

Final Project Report

Prototypical Mask R-CNN for Vehicle Exterior Damage Detection

7COM1039-0901-2025 - Advanced Computer Science Masters
Project

Student ID: 23097240

Name: Zahidul Hoque

Supervisor: Dr Joseph Reddington

[I hereby declare that I have critically proofread and quality checked this report. It is free from grammatical, spelling, and formatting errors.]

MSc FPR Declaration

This report is submitted in partial fulfilment of the requirements for the module “7COM1039-0901-2025-Advanced Computer Science Masters Project”, for the degree of Master of Science in Data Science and Analytics with Advanced Research, at the University of Hertfordshire.

I hereby declare that the work presented in this project and report is entirely my own, except where explicitly stated otherwise. All sources of information and ideas, whether quoted directly or paraphrased, have been properly referenced in accordance with academic standards. I understand that any failure to properly acknowledge the work of others could constitute plagiarism and may result in academic penalties.

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on the university website, provided the source is acknowledged.

Abstract

Automated vehicle damage assessment is transforming the insurance sector at a rapid pace. This automation promises to replace subjective manual surveys with robust and rational solutions. However, deploying computer vision in the vehicle damage detection domain faces challenges when it comes to real-world usage. Vehicle damage image datasets are often imbalanced, and apparent damage categories often share closely resembling visual features. Traditional object detectors relying on linear classification heads often fail to separate these confusing classes and are prone to frequent damage types, which leads to unpredictable automated estimates.

This research aims to overcome these limitations by building a Prototypical Mask R-CNN hybrid architecture that integrates the instance segmentation aptitudes of Mask R-CNN with the generalisation power of Metric Learning, heavily inspired by Meta R-CNN (Xiaopeng Yan *et al.*, 2019). The primary goal is to make a model that learns feature representations for each damage category so that novel damage types can be distinguished.

To achieve this, we have developed a Prototypical Mask R-CNN. We have modified the standard Mask R-CNN architecture by replacing the fully connected classification layer with a Squared Euclidean Prototypical Predictor. Features and learnable class prototypes are projected to the prototypical network in this approach. Classification is determined by minimising the distance between a region and its class centroid rather than learning a separating hyperplane. The model was trained on the publicly available Car Damage Dataset (CarDD).

Experimental results imply that the proposed architecture creates a structured embedding space. Analysis of the latent features reveals that the model successfully learned orthogonal representations for all damage categories in the vector space, which effectively eliminates inter-vehicle damage class confusion. The model achieved a Mean Average Precision (mAP) of 0.319 and AP@50 of 0.630. However, critical performance discrepancy has also been observed between large damage sizes and small Damage sizes, indicative of a bottleneck in the newly developed model.

This study confirms that Prototypical Heads offer training stability and feature separability for novel vehicle exterior damage detection.

Keywords: Computer Vision, Deep Learning, Instance Segmentation, Prototypical Network, Metric Learning, Car Damage Dataset (CarDD), Class Imbalance.

Contents

MSc FPR Declaration

Abstract

Chapter 1: Introduction	1
1.1 Problem Statement	1
1.2 Standalone Convolutional Neural Network limitations	1
1.3 Research Objectives	2
1.4 Research Contributions and Novelty	2
1.5 Project Scope and Feasibility	3
1.6 Report Structure	3
Chapter 2: Literature Review.....	4
2.1 Faster R-CNN.....	4
2.2 Instance Segmentation and Feature Hierarchy.....	5
2.2.1 Feature Pyramid Networks (FPN)	5
2.2.2 Mask R-CNN: Aligning Features	6
2.2.3 The Decoupled Mask Branch	6
2.3 Addressing Class Imbalance via Geometric Constraints	7
2.4 The Geometric Advantage of Metric Learning.....	7
2.5 Metric Learning with Learned Prototypes for Classification.....	8
2.5.1 Inspiration from Prototypical Networks	8
2.5.2 Influence of Meta R-CNN: Remodelling the Predictor Head	8
2.5.3 Synthesis of Proposed Approach.....	8
Chapter 3: Car Damage Dataset Analysis.....	10
3.1 Dataset Overview	10
3.2 Class Distribution.....	11
3.3 The Log-Space Transformation	12
3.4 Scale Variance and the Challenge of Multi-Scale Detection	13
Why Square Root?.....	13
3.5 Photographer Bias	15
Chapter 4: Methodology and Architecture	17
4.1 Architecture Overview	17
4.2 Data Preprocessing and Augmentation	17
4.2.1 Photometric Transformation and Normalisation	17
4.2.2 Geometric Augmentation and Spatial Consistency	18
4.3 Model Architecture	18
4.3.1 Backbone and Feature Pyramid Network.....	18
4.3.2 Region Proposal Network.....	19

4.3.3 The Prototypical Classification Head	19
4.3.4 Box and Mask Heads	19
4.4 Evaluation Metrics	21
4.4.1 COCO Mean Average Precision (mAP).....	21
4.4.2 AP@50 (PASCAL VOC Standard).....	21
4.4.3 Prototype Separability	21
Chapter 5: Experimental Results	22
5.1 Training Dynamics and Convergence	22
5.2 Quantitative Performance (mAP).....	23
5.3 Detailed Metrics Evaluation.....	24
5.4 Feature Space Analysis: The Geometry of Learned Prototypes.	25
5.4.1 Geometric Derivation	26
5.4.2 Interpretation of the Heatmap Values.....	26
Chapter 6: Conclusion and Future Work	28
6.1 Conclusion.....	28
6.2 Future Work	28
References.....	29
Appendix.....	32

Table of figures

Figure 1 Region Proposal Network architecture (Ren et al., 2015) ----- 4

Figure 2 Building block of a featurized image pyramid merged by addition (Lin et al., 2017) 5

Figure 3 Mask R-CNN framework (He et al., 2017) ----- 6

Figure 4 Vehicle Damage class distribution with instance count -----11

Figure 5 Aspect Ratio Variance -----12

Figure 6 Damage size Distribution ($\sqrt{\text{Area}}$) -----14

Figure 7 Spatial Distribution of Damages -----15

Figure 8 Prototypical Mask R-CNN architecture-----20

Figure 9 Training Loss Curve -----22

Figure 10 Validation curve-----23

Figure 11 Prototype Similarity -----25

Chapter 1: Introduction

1.1 Problem Statement

Damage assessment in practice is highly dependent on the individual organisation's experience. One manual adjuster might classify a vehicle exterior damage as a simple "Dent" while another organisation might classify the same damage as a "Structural component damage". This disagreement leads to inconsistent insurance claim standards and regulatory disputes (Shaoqing Ren *et al.*, 2015; Deniz Cultu *et al.*, 2022).

Moreover, the manual review process is often slow and plagued by high claim volumes. This creates an atmosphere where fraudulent claims can slip through undetected. This lack of a consistent visual audit trail makes systemic fraud detection difficult. In an era of instant digital services, this latency is the primary driver of customer churn. Research by McKinsey & Company indicates that the policyholders who submit an insurance claim are likely to switch insurers within 12 months due to dissatisfaction with the speed and lack of consistency of the settlement process (Deniz Cultu *et al.*, 2022).

According to research by Facebook, Computer Vision models offer the promise of manual inspection omission, where users can upload images and get a thorough repair estimate (Tsung-Yi Lin *et al.*, 2018). Nevertheless, the system must be designed in a way that can be used for more than just detecting the presence of certain damage types for specific scenarios, for this approach to be commercially feasible. It must do segmentation that looks for defects to calculate and distinguish between light scratches and structural cracks and operate robustly across thousands of car models and lighting conditions to ensure robustness of the system architecture (Kalpesh Patil *et al.*, 2017).

1.2 Standalone Convolutional Neural Network limitations

Standard Object Detection models, such as the Faster R-CNN are typically optimised on benchmarking datasets like MS-COCO or Pascal VOC (Mark Everingham *et al.*, 2010; Tsung-Yi Lin *et al.*, 2015). These datasets are curated to maintain a balanced class distribution and consistent object aspect ratios that allow models to learn features across all categories.

In contrast, vehicle damage data represents a critical example for deep learning, which is characterised by a severe "Long-Tail" distribution in the CarDD dataset. The dataset follows a Zipfian power law, as mentioned in the CarDD benchmark (Ziwei Liu *et al.*, 2019; Xinkuang Wang, Wenjing Li and Zhongheng Wu, 2023).

CarDD damage classes distribution can be distinguished into two classes:

1. Majority Classes: Exterior vehicle damage classes like "Scratches" and "Dents" occur with high frequency, which accounts for the vast majority of claims that resemble the head of the "Long-tail" distribution of damage classes.
2. Minority Classes: Structural damages like "Glass Shatters", "Flat Tires" or "Lamp Broken" are comparatively rare, which resembles the tail of the long-tailed distribution.

When a standard Convolutional Neural Network (CNN) is trained on such imbalanced data using standard Cross-Entropy Loss, it suffers from gradient dominance. As described by Tsung-Yi Lin *et al.* in their analysis of Focal Loss, the gradients from the majority examples overshadow the learning signal from the minority examples. Consequently, the network converges to a local minimum where it prioritizes the majority class to minimize the global error rate (Tsung-Yi Lin *et al.*, 2018).

A fundamental study about large-scale long-tailed recognition in an open world by Ziwei Liu et al. demonstrates that in long-tailed distribution scenarios, the L_2 normalisation of the weight vectors for the majority classes grows significantly larger than that of the minority classes (Ziwei Liu *et al.*, 2019). Based on the work of Tsung-Yi Lin et al. and Ziwei Liu et al. this statistical bias is catastrophic in a practical insurance claiming context, which often leads to missing critical damage classifications.

1.3 Research Objectives

The principal objective of this project is to study and develop a deep learning architecture that decouples representation learning from class frequency. We aim to move beyond standard supervision towards Metric Learning, where the model learns to map images to a vector space where distance corresponds to semantic similarity.

1. Research Question 1: Can a metric learning approach like Prototypical Networks be integrated into Mask R-CNN to work on novel damage classes?
2. Research Question 2: How does the integration of a squared Euclidean Prototypical Head within a Mask R-CNN Framework influence the geometric arrangement of the latent feature space?
3. Research Question 3: What is the measured improvement in F1 score, and Average Recall (AR) achieved by prototypical Mask R-CNN on minority classes and does the architecture effectively mitigate the bias inherent in long-tail class distributions?

1.4 Research Contributions and Novelty

This research implements architectural changes in Mask R-CNN architecture that result in going away from hyperplane-based object detection pipelines to a prototypical network-based metric learning approach. While the Mask R-CNN framework serves as the foundational baseline for this study, this project focuses on integrating Metric Learning into a fully supervised instance segmentation. Based on the work of Shaoqing Ren et al. and Kaiming He et al., Faster R-CNN and Mask R-CNN rely on Linear Classification Heads, which attempt to separate target classes using flat hyperplanes. This hyperplane-based classification approach struggles when vehicle damage classes share significant visual overlap (Shaoqing Ren *et al.*, 2015; Kaiming He *et al.*, 2017).

This project replaces the linear classifier with a Squared Euclidean Prototypical Predictor. The network learns to map features to a prototype centroid instead of learning decision boundaries. This converts the classification task from a partitioning problem into a nearest neighbour problem within the latent space (Jake Snell, Kevin Swersky and Richard Zemel, 2017).

This project also proposes a structural geometric solution to the class imbalance issue. The Prototypical Head clusters data around a centroid rather than pushing a decision boundary. The gradient volume from the majority classes does not crowd out the minority classes. The prototype for a rare class maintains its distinctive position in the embedding space regardless of the frequency of other classes (Jake Snell, Kevin Swersky and Richard Zemel, 2017).

1.5 Project Scope and Feasibility

The feasibility of this research is established through the application of the CarDD Dataset benchmark, which is publicly available and memory optimisation, such as Automatic Mixed Precision for utilising the NVIDIA RTX 3060 (12GB) environment. The primary contribution of this research is the combination of Metric Learning to solve damage class imbalance. Given the time constraints of the research project, to align with the computational constraints of the available Single GPU, the scope is restricted specifically to single-frame analysis rather than multi-frame video processing. This allows us to research the Prototypical Mask R-CNN architecture for solving issues targeted for the insurance industry. Furthermore, single-frame analysis has ensured that the model remains lightweight enough for potential edge deployment.

1.6 Report Structure

The remainder of this report is organised as follows:

1. Chapter 2 (Literature Review): A thorough analysis of the evolution of Instance Segmentation and the theoretical foundations of Metric Learning.
2. Chapter 3 (Car Damage Dataset Analysis): A justification of the architectural decisions based on aspect ratio, scale variance and computer vision characteristics through statistical analysis of the Car Damage Dataset (CarDD).
3. Chapter 4 (Methodology and Architecture): The implementation of the Prototypical Mask R-CNN and the mathematical derivation.
4. Chapter 5 (Experimental Results): Validation of the Prototypical Mask R-CNN model through industry standard metrics for COCO and the prototype distance metric.
5. Chapter 6 (Conclusion and Future Work): Summary of the research and discussion of the limitations, and proposes a future integration with VLA frameworks

Chapter 2: Literature Review

2.1 Faster R-CNN

The R-CNN framework reached maturity with the introduction of Faster R-CNN by (Shaoqing Ren *et al.*, 2015). Prior to this innovation, state-of-the-art networks such as Fast R-CNN by (Ross Girshick, 2015) relied on algorithms like Selective Search to generate region proposals. These algorithms created a significant computational bottleneck while running on the CPU. This issue effectively prevented real-time inference.

(Shaoqing Ren *et al.*, 2015) resolved this bottleneck by introducing the Region Proposal Network. The Region Proposal Network is a fully convolutional network that shares the same convolutional features of the backbone as the detection network, which effectively enables nearly computational cost-free region proposals. The Region Proposal Network operates by sliding a small spatial window of the matrix ($n \times n$) over the convolutional feature map. The network simultaneously predicts multiple region proposals based on k fixed reference boxes at each sliding window location. These reference boxes are fundamental to handling the scale variance inherent in object detection without requiring an image pyramid. In the specific configuration proposed by (Shaoqing Ren *et al.*, 2015), the system utilises three scales and three aspect ratios (1: 1,1: 2,2: 1), yielding $k = 9$ anchors at every sliding position.

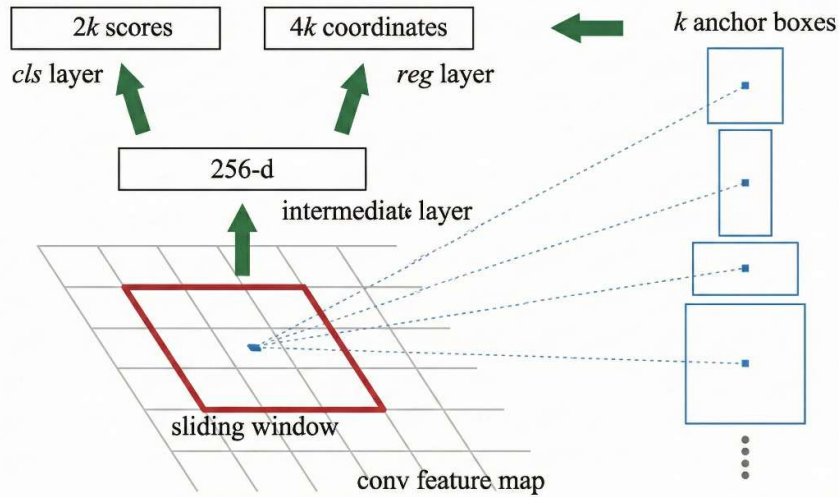


Figure 1 Region Proposal Network architecture (Shaoqing Ren *et al.*, 2015)

This architecture establishes the first stage of the detector. By filtering out the majority of the background regions using Non-Maximum Suppression, the RPN passes a set of proposals to the second stage. The Faster R-CNN then uses Region of Interest (RoI) Pooling to extract a certain-sized feature vector from the variable-sized proposals, which eventually leads to performing specific class classification and precise bounding box regression (Shaoqing Ren *et al.*, 2015).

This model includes first proposing and then verifying. As a result, it provides superior localisation accuracy required for identifying vehicle exterior damage features. Furthermore, (Shaoqing Ren *et al.*, 2015) introduced a training algorithm to allow the RPN and Fast R-CNN to share convolutional weights, which ensures the proposal generation is informed by the same features used for classification.

2.2 Instance Segmentation and Feature Hierarchy

While bounding boxes provide localisation of objects, they are fundamentally ill-suited for characterising smaller and irregular object shapes. A standard bounding box drawn around a thin diagonal scratch will consist largely of noise pixels belonging to the healthy vehicle exterior surface rather than the damage itself. To address this, we must utilise Instance Segmentation, specifically the Mask R-CNN framework by (Kaiming He *et al.*, 2017), which supports comparatively robust feature extraction.

2.2.1 Feature Pyramid Networks (FPN)

Before segmentation can occur, the image must be processed into a feature representation for the CNN to classify in the mask R-CNN architecture to work. However, a research by Tsung-Yi Lin *et al.* Standard Convolutional Neural Networks (CNNs) face an inherent trade-off. Deep layers have strong semantic information knowing what an object is, but low spatial resolution resulting in losing where it is (Tsung-Yi Lin *et al.*, 2017).

To resolve this, Tsung-Yi Lin *et al.* introduced the Feature Pyramid Network (FPN). Feature Pyramid Network exploits the inherent multi-scale pyramidal hierarchy of deep convolutional networks to construct feature pyramids with marginal extra cost (Tsung-Yi Lin *et al.*, 2017).

FPN is comprised of three main components:

1. Bottom-Up Pathway: The standard feed-forward computation of the backbone CNN such as ResNet-50 computes a feature hierarchy consisting of feature maps at several scales with a scaling step of 2.
2. Top-Down Pathway: The system takes the features from the deepest layer of the network and increases their size by a factor of 2 to match the resolution of the shallower layer below it.
3. Lateral Connections: The upscaled map is merged with the corresponding bottom-up map via element-wise addition. A 1×1 convolution is applied to the bottom-up map to match channel dimensions.

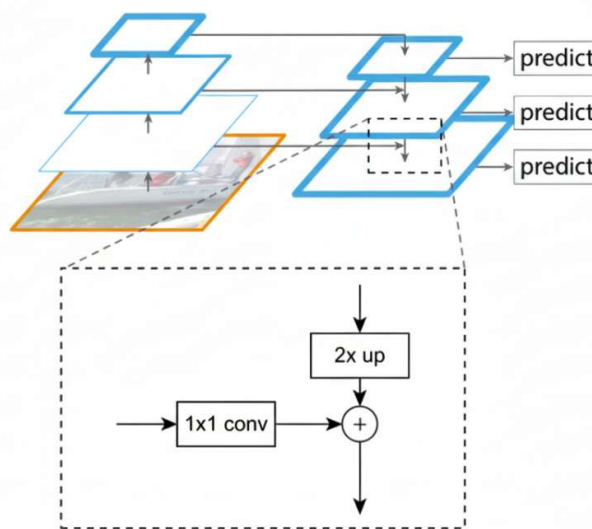


Figure 2 Building block of a featurized image pyramid merged by addition (Tsung-Yi Lin *et al.*, 2017)

The result is a feature pyramid that possesses strong semantics and appropriate resolution. Tsung-Yi Lin et al. demonstrated that using Feature Pyramid Network within a Faster R-CNN system significantly improved Average Precision (AP), especially for small objects, by assigning Region of Interest (RoI) proposals to specific pyramid levels based on their scale. This method ensures that small scratches are analysed using high-resolution feature maps, while large dents are analysed using lower-resolution, semantically broad maps (Tsung-Yi Lin *et al.*, 2017).

2.2.2 Mask R-CNN: Aligning Features

Kaiming He et al. extended the Faster R-CNN architecture to create Mask R-CNN. The critical innovation required for this extension was solving the problem of misalignment (Kaiming He *et al.*, 2017).

The region of interest pooling operation was used to extract features for a specific region in the Faster R-CNN architecture. If a region proposal has decimal coordinates, the RoI pooling operation rounds these to the nearest integers to fit the feature map grid. While this slight shift is negligible for drawing a bounding box, it destroys the pixel-wise alignment which is required for accurate segmentation.

Kaiming He et al. introduced Region of Interest Alignment to fix this. RoIAlign removes the harsh quantisation of RoIPool. It uses bilinear interpolation to compute the exact values of the input features at four regularly sampled locations within each RoI bin. This ensures that the extracted feature maps align perfectly with the original image pixels. The authors noted that this minor change improved mask accuracy by 10% to 50% which proves essential for high fidelity segmentation (Kaiming He *et al.*, 2017).

2.2.3 The Decoupled Mask Branch

With aligned features available, Mask R-CNN adds a parallel branch to the network head. While the first two branches predict the class label and bounding box offsets, the third branch, which is a Fully Convolutional Network (FCN), predicts a binary mask for each Region of Interest (RoI).

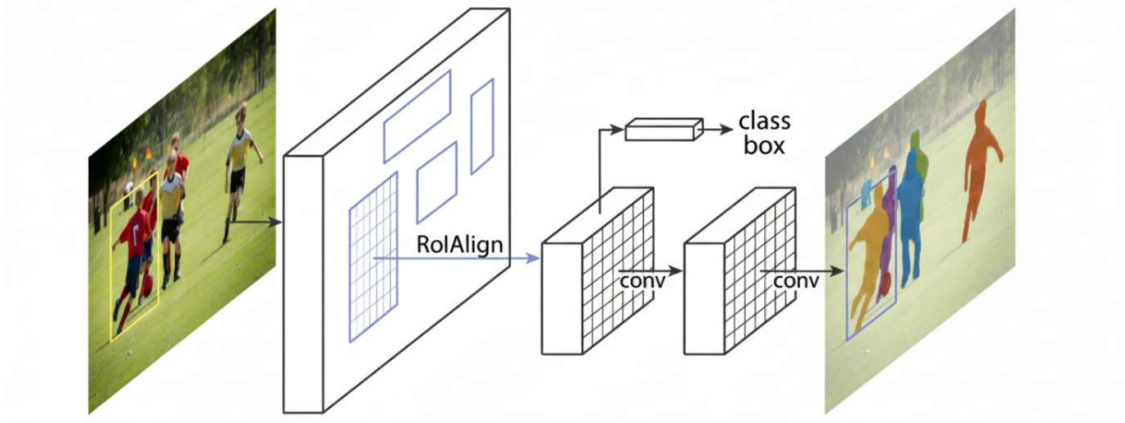


Figure 3 Mask R-CNN framework (Kaiming He et al., 2017)

Kaiming He et al. utilised a decoupled approach to segmentation. Rather than predicting a multi-class mask where classes compete for every pixel via a SoftMax, the mask branch generates K binary masks independently that represent each of the K classes. The system relies on the classification branch to select the correct mask (Kaiming He *et al.*, 2017). For example, if the classifier predicts “scratch” the model selects the specific “scratch Mask” and ignores other damage classes. This decoupling simplifies the training process and avoids competition between target classes.

2.3 Addressing Class Imbalance via Geometric Constraints

There are thousands of examples of common damage types like “Dents” or “Scratches”, but very few examples of rare types such as “Shattered glass” or “Flat tires” within a dataset like CarDD (Xinkuang Wang, Wenjing Li and Zhongheng Wu, 2023). This damage class imbalance poses a significant challenge for conventional deep learning models for the detection task.

Mask R-CNN has a linear classification head ($W^T x + b$) optimised via Cross Entropy loss. As noted by Kaiming He et al., these parametric classifiers are sensitive to class frequency. During training, the weight vector W is updated to minimise the global error (Kaiming He *et al.*, 2017). In a statistically imbalanced scenario, the gradients originating from the majority class examples dominate the optimisation process. This causes the decision boundary to shift, in order to prioritise the majority class to satisfy the global loss function and often sacrificing the accuracy of rare classes. This phenomenon results in a model that performs well on common defects but is unreliable for novel damage types.

2.4 The Geometric Advantage of Metric Learning

To combat this frequency-driven bias without resorting to complex data weighting alteration, this project leverages the geometric properties of Metric Learning. Unlike linear classifiers that attempt to partition the feature space with hyperplanes, Prototypical Networks classify based on the distance to a learned centroid (Jake Snell, Kevin Swersky and Richard Zemel, 2017).

This approach introduces a geometric regularisation effect. The prototype for a rare class is represented by a learnable vector situated on a unit hypersphere. Even with a few training examples, this prototype occupies a distinct region in the embedding space. As long as the network learns to map the visual features of a certain rare damage class to that specific region, the classification will be robust. The presence of the majority of damage class examples primarily influences the position of that particular damage class prototype. However, It does not necessarily crowd out or suppress the region for the rare damage class prototype, provided the embedding dimension is sufficiently high to allow for such separation (Jake Snell, Kevin Swersky and Richard Zemel, 2017).

By normalising both the RoI features and the learned prototypes to lie on a unit hypersphere, we eliminate magnitude variations. This ensures that the model classifies based purely on the shape of the feature representation. This procedure makes the architecture robust to the issue of long-tailed distribution and reduces the adverse effects of class imbalance inherent in the CarDD dataset (Jake Snell, Kevin Swersky and Richard Zemel, 2017; Xinkuang Wang, Wenjing Li and Zhongheng Wu, 2023).

2.5 Metric Learning with Learned Prototypes for Classification

The final linear layer in Faster R-CNN learns by utilising a separating hyperplane for each class. While effective for data-rich classes, this approach often struggles with the long-tailed problem observed in car damage datasets, where rare classes have insufficient examples to define a robust linear decision boundary (Ziwei Liu *et al.*, 2019). To address this limitation, we turn to Metric Learning, which aims to learn a structured embedding space where similar instances are clustered together (Jiawei Wu, Wenhan Xiong and William Yang Wang, 2019).

2.5.1 Inspiration from Prototypical Networks

A foundational inspiration for this approach comes from Prototypical Networks (Jake Snell, Kevin Swersky and Richard Zemel, 2017), originally proposed for few-shot learning. The core idea is that there exists an embedding space where data points from the same class cluster around a single representative prototype. In their work, this prototype was dynamically computed as the mean of a few support examples. Classification was then performed by finding the nearest class prototype to a query image using Squared Euclidean Distance. (Jake Snell, Kevin Swersky and Richard Zemel, 2017) demonstrated that this distance metric significantly outperformed the cosine similarity.

2.5.2 Influence of Meta R-CNN: Remodelling the Predictor Head

The translation of these metric learning concepts into the distinctive architecture of Mask R-CNN was heavily influenced by Meta R-CNN (Xiaopeng Yan *et al.*, 2019). In their work on low-shot instance segmentation, Xiaopeng Yan *et al.* identified that Mask R-CNN heads obscure low-shot classes by merging multiple object information with complex backgrounds into fixed linear weights. They argued that to generalise to rare categories, the network must disentangle the class representation from the specific RoI feature extraction.

Xiaopeng Yan *et al.* then proposed a methodology called Predictor head Remodelling, where the standard classification head is replaced by a mechanism that interacts RoI features with distinct class attentive vectors (Xiaopeng Yan *et al.*, 2019). This project adopts this specific architectural philosophy. This study includes remodelling the predictor head to explicitly compare RoI features against distinct class representations. While Meta R-CNN infers these vectors dynamically for few-shot tasks, we use them as static, learnable Prototypical Vectors to stabilise training on the long-tailed vehicle damage class distribution of CarDD. This structural change forces the network to learn features that align geometrically with their class prototype.

2.5.3 Synthesis of Proposed Approach

This project synthesises the advancements in instance segmentation of Mask R-CNN with the architectural remodelling of Meta R-CNN and the distance-based objectives of Prototypical Networks.

We utilise the ResNet50-FPN backbone for robust multi-scale feature extraction and the Mask R-CNN head with RoIAlign for pixel-perfect segmentation. However, the key modification lies in replacing the standard linear box classifier with our custom Prototypical Box Predictor:

- Learned Prototypes: A set of static prototype vectors is defined as learnable parameters, one for each damage class. Following the intuition of (Xiaopeng Yan *et al.*, 2019) These act as the ideal damage class representative templates within the embedding space.
- Hypersphere Projection: Both the Region of Interest features and the learned prototypes are L2-normalised. This projects all vectors onto a unit hypersphere, resulting in emphasising angular separation over magnitude.
- Metric-Based Classification: The model classifies a detected region by calculating the Squared Euclidean Distance to all class prototypes (Jake Snell, Kevin Swersky and Richard Zemel, 2017). The negative distances are then passed through a SoftMax function to yield probabilities.

By replacing the dot product similarity with a distance-based metric on a constrained hypersphere, we ensure that novel damage classes maintain orthogonal representations in the latent space. This prevents the majority of classes from dominating the decision boundary purely through feature magnitude, thereby improving the prototypical detector's classification power for vehicle exterior damage detection.

Chapter 3: Car Damage Dataset Analysis

A statistical analysis of the Car Damage Dataset (CarDD) has been conducted before model design. This analysis is to justify the specific architectural components for this project, which is primarily based on the work of Jiawei Wu et al., Kaiming He et al., and Tsung-Yi Lin et al. (Kaiming He *et al.*, 2017; Tsung-Yi Lin *et al.*, 2017; Jiawei Wu, Wenhan Xiong and William Yang Wang, 2019).

3.1 Dataset Overview

The Car Damage dataset of (Xinkuang Wang, Wenjing Li and Zhongheng Wu, 2023) serves as the standard for evaluating fine-grained Vehicle damage recognition in the automotive domain. It is the first large-scale public benchmark dedicated to high-resolution car damage instance segmentation. The dataset comprises 4000 high-resolution images. This heterogeneous collection strategy is critical for the research as it provides images of damaged vehicles from real-world conditions. It captures vehicles in uncontrolled environments with varying lighting and camera angles, which closely mimic the raw photographic data uploaded during real-world insurance claim scenarios (Xinkuang Wang, Wenjing Li and Zhongheng Wu, 2023).

The dataset provides dense supervision with over 9000 pixel-level annotated instances. These are categorised into six Damage classes representing the most common exterior damages:

1. Dent
2. Scratch
3. Crack
4. Glass Shatter
5. Tire Flat
6. Lamp Broken

To ensure comparability with existing benchmarks, we adhere to the official data partitioning protocol defined by Xinkuang Wang et al. The dataset is divided into a training set of 2,817 images (70.4%), a validation set of 810 images (20.25%), and a test set of 373 images (9.35%) (Xinkuang Wang, Wenjing Li and Zhongheng Wu, 2023).

3.2 Class Distribution

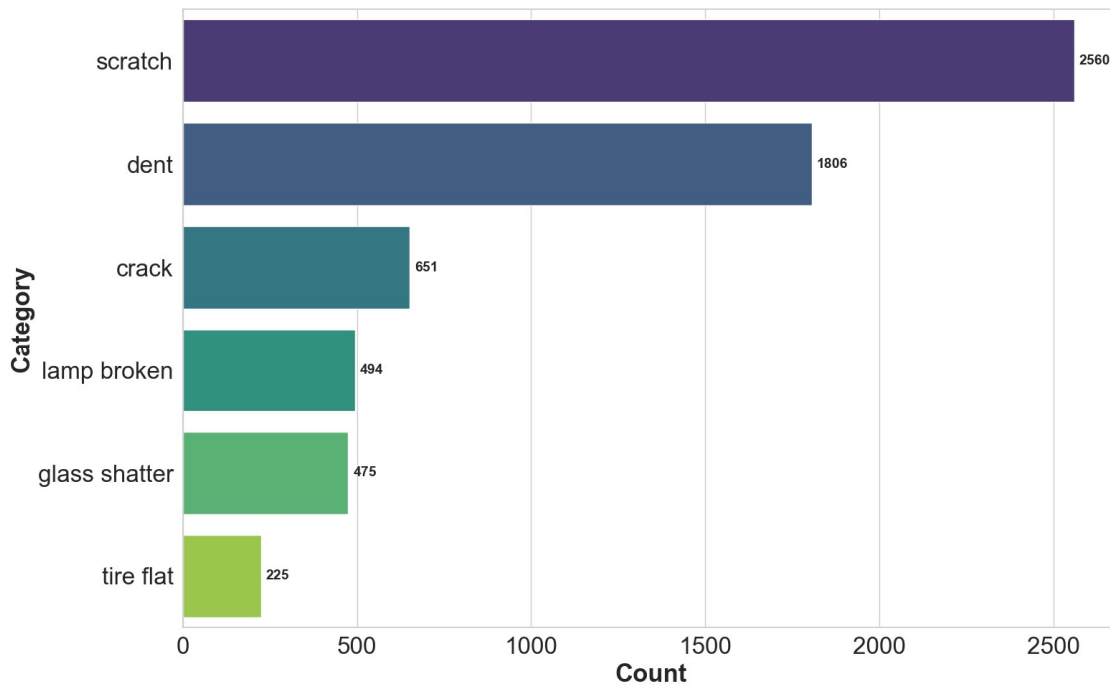


Figure 4 Vehicle Damage class distribution with instance count

We performed a statistical analysis of the annotation frequency across the training set to understand the damage class characteristics in the Car Damage Dataset (CarDD). As illustrated in Figure 4, the dataset exhibits a long tail distribution (Ziwei Liu *et al.*, 2019). This phenomenon is abundant in real-world industrial applications where minor defects occur frequently, but severe or catastrophic failures are naturally rare.

The distribution reveals a distinct separation between the “Head” and “Tail” distribution of vehicle damage classes:

- The Head: The dataset is heavily dominated by cosmetic damage. “Scratch” ($n = 2560$) and “Dent” ($n = 1806$) constitute the majority of the distribution. This reflects the reality of vehicle incidents, which is that minor scrapes and parking bumps are far more common than major accidents.
- The Tail: Structural and safety-critical damages reside in the rarest class, such as “Tire Flat” ($n = 225$).
- The Imbalance Ratio: The disparity is quantifiable via the imbalance ratio. Comparing the most frequent class to the least frequent, we observe a ratio of $11.4 : 1$, which is considered a “severe imbalance” in deep learning terms (Xinkuang Wang, Wenjing Li and Zhongheng Wu, 2023).

This long-tailed distribution is a problem for standard Mask R-CNN as it uses a linear classification head optimised by Cross-Entropy Loss. The model minimises its global error by becoming biased towards the majority class because that is the safest way to lower the total loss. This leads to high accuracy on common classes, but near-zero recalls on rare target classes (Jake Snell, Kevin Swersky and Richard Zemel, 2017; Kaiming He *et al.*, 2017; Ziwei Liu *et al.*, 2019).

This analysis provides the primary architectural justification for our adoption of Prototypical Networks. By replacing the standard linear classifier with a Prototypical Head, we fundamentally change the learning objective from frequency to geometry. In the Prototypical approach, the model learns a centroid for each class in the embedding space (Jake Snell, Kevin Swersky and Richard Zemel, 2017).

Here, the “Tire Flat” centroid and the “Scratch” centroid are treated as two equally important target points in the vector space, despite “Scratch” having more data points clustering around it (Jake Snell, Kevin Swersky and Richard Zemel, 2017). This prevents the majority of the class from crowding out the minority class. As long as the model learns to map the visual features of a certain damage class to the correct geometric region, it will classify it correctly regardless of how many scratches exist elsewhere in the dataset.

3.3 The Log-Space Transformation

We examined the Aspect Ratio (AR) of the ground truth bounding boxes.

Standard linear analysis ($AR = Width/Height$) is statistically flawed for investigating variance due to asymmetry. For example, a “tall” object with a ratio of 1:2 (0.5) and a “wide” object with a ratio of 2:1 (2.0) are geometrically inverse to each other. However, on a linear scale, the tall object is 0.5 units away from a square (1.0), while the wide object is 1.0 unit away. This bias makes it impossible to accurately calculate the mean or variance of the distribution (Luciano Da Fontoura Costa. and Roberto Marcond Cesar Jr., 2010).

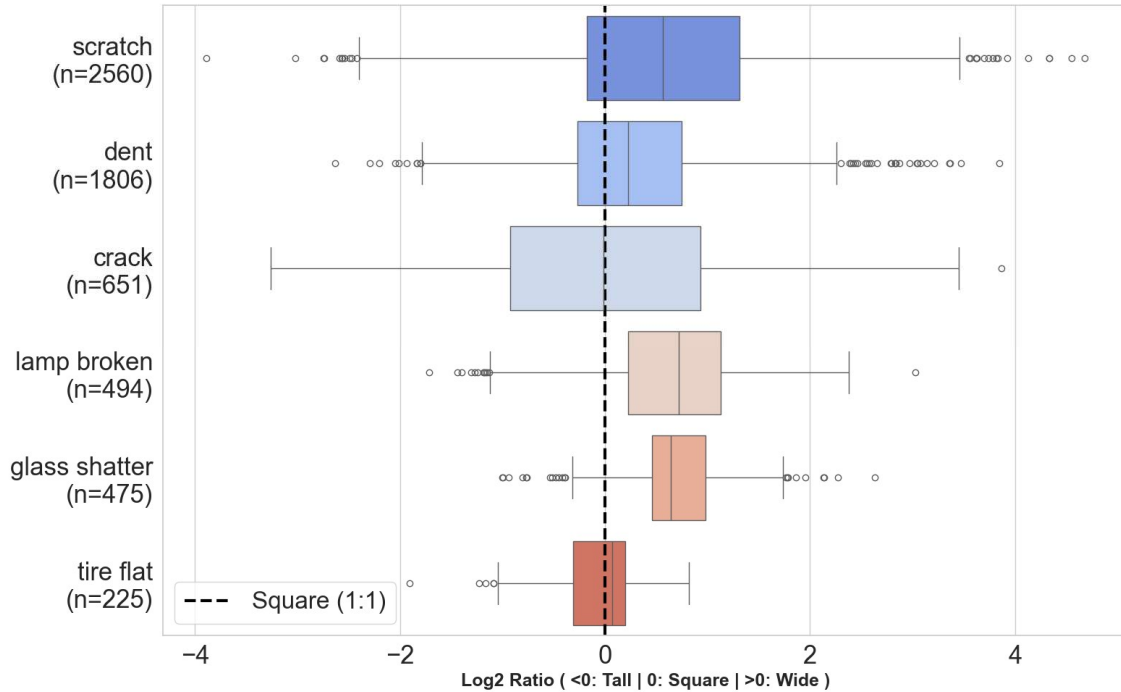


Figure 5 Aspect Ratio Variance

To combat this, we have utilised a Log2 Transformed Aspect Ratio:

$$Aspect\ Ratio_{log} = \log_2(W/H)$$

This formula is based on the scientific paper by Luciano Da Fontoura Costa. & Roberto Marcond Cesar Jr. This transformation linearises the ratio space. Here,

- $\log_2(\text{Aspect Ratio}) = 0$; perfect Square (1:1).
- $\log_2(\text{Aspect Ratio}) = -3$; Extreme Verticality (1:8 ratio).
- $\log_2(\text{Aspect Ratio}) = +3$; Extreme Horizontality (8:1 ratio).

The boxplot visualisation in Figure 5 reveals a distinction. “Dents” cluster tightly around 0, indicating that they are generally square. In contrast, “Scratches” shows immense variance with outliers extending from -3 to +3. This confirms that scratches are highly likely to appear as long horizontal, vertical or diagonal lines (Luciano Da Fontoura Costa. and Roberto Marcond Cesar Jr., 2010).

Moreover, the Box Regression Head predicts scale-invariant log-space offsets relative to the anchor boxes in the Mask R-CNN architecture. As defined by Shaoqing Ren et al. the regression target for width (t_w) is calculated as:

$$t_w = \log\left(\frac{w_{pred}}{w_{anch}}\right)$$

The network optimises these log offsets for two critical reasons:

1. Positivity Constraint: Geometric dimensions (width and height) must be strictly positive. The network can output any real number by predicting (t_w) in log space. The exponential function guarantees the resulting width is always positive when inverted during inference, where $w = w_{anch} \cdot e^{t_w}$ (Shaoqing Ren *et al.*, 2015)
2. Scale Invariance: It normalises the error magnitude. According to Dr. Chris Tofallis, Log space regression penalises the relative error, which ensures that small defects are trained with equal importance to large ones (Chris Tofallis, 2009).

3.4 Scale Variance and the Challenge of Multi-Scale Detection

A critical challenge in automated vehicle inspection is Scale Variance. Damage classes can be observed at vastly different magnitudes, ranging from a few pixels to occupying half the image. To measure this variance effectively, we analysed the distribution of object sizes using the square root of the segmentation area ($\sqrt{Area}$).

Why Square Root?

Raw area (pixels²) grows quadratically. A 100×100 Pixel object has 4 times the area of a 50×50 Pixel object, but visually it only appears twice as large. The square root linearises the metric and aligns it with standard benchmarks. We classify objects based on the established MS-COCO definitions (Tsung-Yi Lin *et al.*, 2015):

- Small: less than 32 pixels
- Medium: 32 to 96 pixels.
- Large: More than 96 pixels.

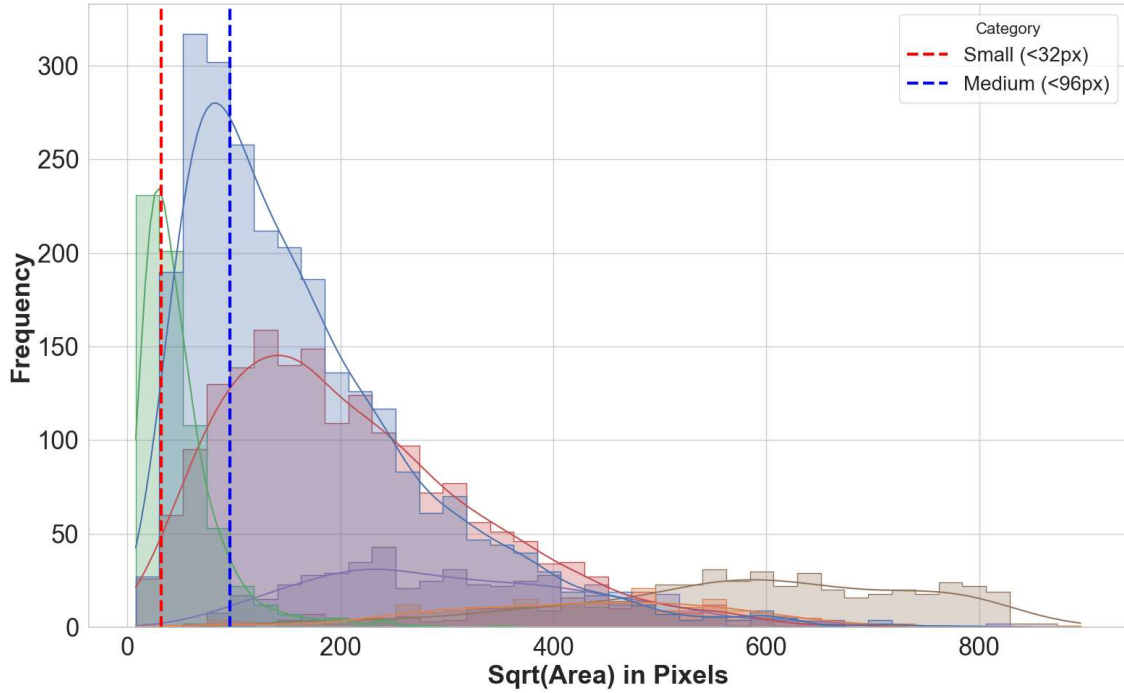


Figure 6 Damage size Distribution ($\sqrt{\text{Area}}$)

As illustrated in Figure 6, the dataset exhibits extreme scale discrepancy:

- A significant portion of defects fall below the 32px threshold, which is indicated by the red dashed line in the figure. These are tiny features relative to the full 1024×1024 Input resolution.
- Conversely, damage categories like “Dent” and “Glass Shatter” span a massive range, with the distribution centred around 100 to 200 pixels.

So, the detection model must simultaneously resolve fine-grained textures and abstract global shapes. This scale difference is difficult for Convolutional Neural Networks to handle using a single feature map.

This statistical evidence provides the definitive justification for utilising a ResNet-50-FPN backbone (Adam Paszke *et al.*, 2019). In a standard deep CNN (like ResNet-50), the image undergoes five stages of down-sampling to extract semantic features. The final feature map has a stride of 32. This means every 32 pixels in the input image is compressed into just a single pixel in the feature map (Li Liu *et al.*, 2020).

So, if there is a small scratch from a cluster that is 24 pixels wide, this object is projected to $24/32 = 0.75$ Pixels in the final layer. So, it effectively becomes noise. By employing a Feature Pyramid Network (FPN), our model preserves sufficient spatial resolution to resolve the defect, allowing the model to detect the “Small” cluster identified in our analysis while still using deep layers to classify large objects (Tsung-Yi Lin *et al.*, 2017).

3.5 Photographer Bias

Finally, we observed the object positioning within the image in the training data. In an ideal dataset, objects would appear uniformly across the image canvas. To test this, we generated a 2D Spatial Heatmap by grouping the centroids of all damage annotations. The X and Y axes represent the pixel coordinates of the image frame, which is (1024×1024) and the colour intensity indicates the density of defect occurrences.

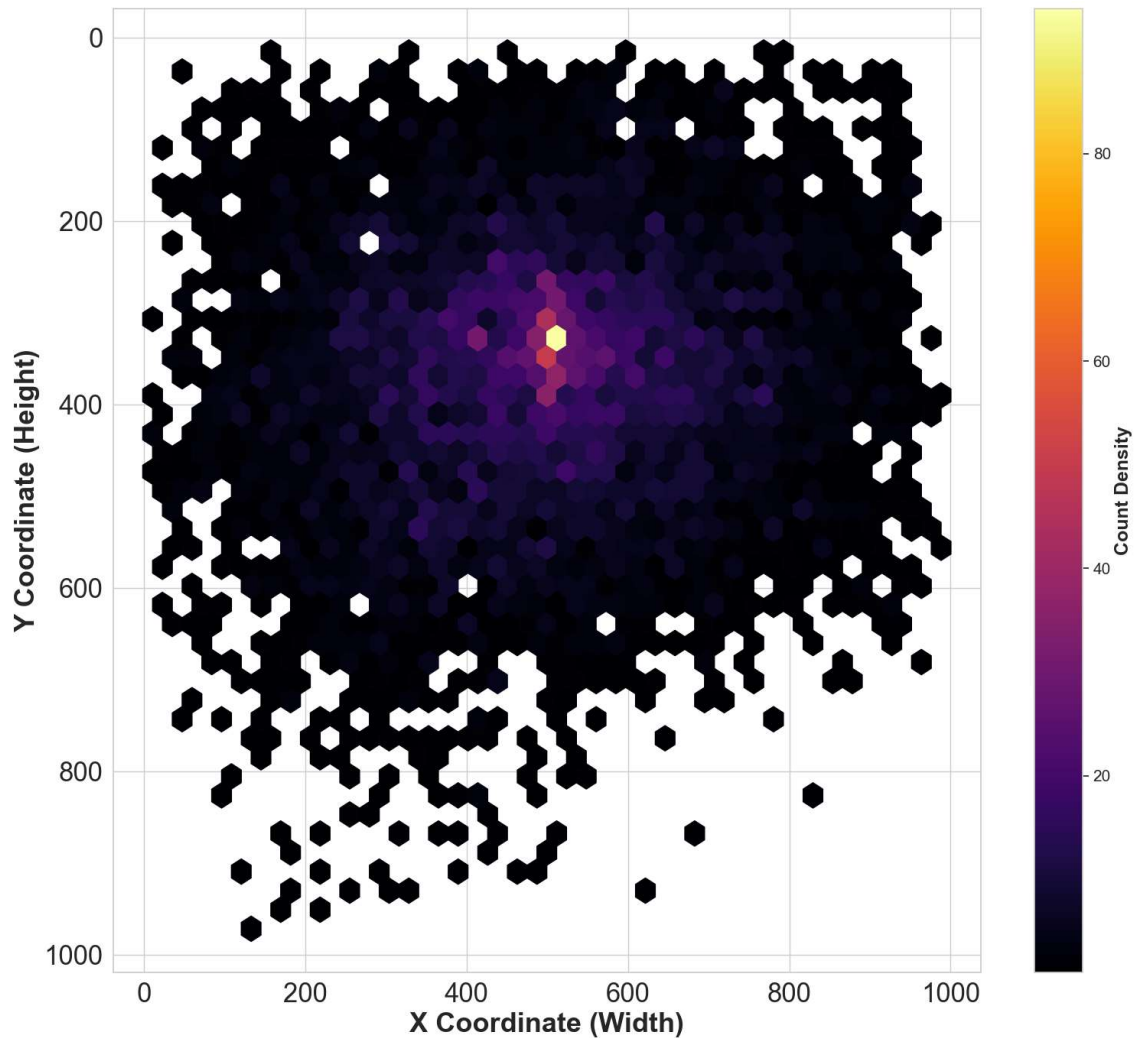


Figure 7 Spatial Distribution of Damages

As visualised in Figure 7, the dataset reveals a pronounced centre bias. This pattern is a manifestation of “Photographer Bias” mentioned by Torralba & Efros in their work about “Unbiased Look at Dataset Bias” (Antonio Torralba and Alexei A. Efros, 2011)

In the context of insurance claims, the photos are taken by humans who have a specific intent, which is to document the damage. When a human takes a photo of a dent, they instinctively frame the damage in the centre of the viewfinder. They rarely take a photo where the damage is cut off at the bottom corner.

This bias poses a significant hazard for Deep Learning. The model might learn to classify in such a way that if a feature is in the centre, it is damaged, and if it is at the edge, then it is background. This phenomenon might lead to poor generalisation in real-world scenarios where damage moves across the frame (Kemal Oksuz *et al.*, 2021).

To counteract this bias issue, we can utilise a Fully Convolutional Network (ResNet-FPN) as the backbone. A convolutional filter that learns to detect the texture of a “scratch” shares its weights across the entire image. If the network learns what a scratch looks like in the centre of the image, the same mathematical operation allows it to detect that scratch at the edge of the image, even if it rarely saw examples there during training (Yann Lecun, Yoshua Bengio and Geoffrey Hinton, 2015).

To further force the model to look at the dark edges of the heatmap, we can implement a Region Proposal Network. The RPN performs a uniform sliding window operation by placing anchor boxes at fixed intervals across every inch of the feature map, which includes corners. It prevents the model from ignoring the edge, resulting in actively counteracting the CarDD dataset’s centre bias attribute (Shaoqing Ren *et al.*, 2015; Xinkuang Wang, Wenjing Li and Zhongheng Wu, 2023).

Chapter 4: Methodology and Architecture

4.1 Architecture Overview

To address the challenge of small-sized damages in the vehicle exterior damage detection, we have implemented an Instance Segmentation framework. The core architecture is built upon the Mask R-CNN, utilising the Region-based Convolutional Neural Network. Moreover, to improve Novel damage class separability among visually similar damage types, the standard linear classification head has been replaced with a Metric Learning-based Prototypical Head.

The project approach is to perform three simultaneous tasks:

1. Object Classification: Identifying the types of damage
2. Bounding Box Regression: Localising the damage with coordinates.
3. Instance Segmentation: Generating a pixel-perfect mask of the damaged area.

4.2 Data Preprocessing and Augmentation

The image data from the Car Damage Dataset presents some challenges, which include uncontrolled lighting environments and the reflective nature of vehicle surfaces. To ensure the neural network generalises well and does not overfit specific features, data augmentation was implemented within the Damage classes of the Car Damage Dataset.

4.2.1 Photometric Transformation and Normalisation

The visual appearance of vehicle damage is heavily affected by environmental lighting and paint finish. A dent on a matte grey bumper appears significantly different from the same dent on a glossy grey bumper due to specular highlights and shadows. To force the model to learn the structural features of damage rather than colour-specific patterns, the following photometric operations are applied:

1. Tensor Conversion: Input images are converted from integer pixel values to floating-point tensors. This puts the data in the correct format for GPU acceleration and automatic differentiation (Adam Paszke *et al.*, 2019).
2. Colour Jittering: Based on the survey about Image data Augmentation for Deep Learning by Connor Shorten & Taghi M. Khoshgoftaar, a Stochastic colour transformation is applied (Connor Shorten and Taghi M. Khoshgoftaar, 2019). During model training with the following parameters:
 - Brightness (± 0.2): Simulates varying exposure levels, ensuring the model can detect damage in both overexposed (direct sunlight) and underexposed (garage/shadow) scenarios.
 - Contrast (± 0.2): Critical for detecting subtle features like “scratches” or “cracks” against low-contrast backgrounds.
 - Saturation (± 0.2): Prevents the model from over-relying on specific colour intensities, making it robust to different sensor colour profiles and paint fading.

4.2.2 Geometric Augmentation and Spatial Consistency

Given the limited size of the dataset, geometric augmentation is implemented based on the survey by Connor Shorten & Taghi M. Khoshgoftaar to artificially expand the training set and reduce bias (Connor Shorten and Taghi M. Khoshgoftaar, 2019).

Here, A horizontal flip is applied with a probability of $p = 0.5$. This transformation is particularly effective for vehicle analysis due to the lateral symmetry of cars; a dent on the left fender is morphologically similar to a dent on the right fender. This effectively doubles the diversity of the training data.

Moreover, Spatial labels are transformed synchronously with the image. The annotations have dynamically been recalculated when an image is flipped. The binary masks are also flipped using a library function of torchvision named “`torchvision.transforms.functional.hflip`” to ensure the pixel-level ground truth aligns perfectly with the new image orientation (Adam Paszke *et al.*, 2019).

This synchronisation prevents the model from identifying noisy labels by ensuring the model is not supervised using the original coordinates when the input image has been transformed.

4.3 Model Architecture

The proposed system is built upon the Mask R-CNN framework, which is a two-stage detector. While conventional detectors rely on linear classifiers, this newly developed architecture integrates a Metric Learning component to handle the visual complexity of car damage. The architecture operates through some evident stages which include feature extraction, region proposal, prototypical classification, bounding box regression and mask segmentation.

4.3.1 Backbone and Feature Pyramid Network

The foundation of the model is a ResNet-50 backbone, pre-trained on the ImageNet dataset (Adam Paszke *et al.*, 2019). This component is responsible for transforming raw image pixels into abstract feature representations.

A Feature Pyramid Network is attached to the backbone to address the scale variation problem in the Car Damage Dataset, where damage can range from a microscopic damage class to a massive dent (Tsung-Yi Lin *et al.*, 2017).

According to the findings of Kaiming He *et al.*, ResNet-50 usually produce low-resolution, semantic features at the end. This workflow is good for classifying a whole car, but bad for detecting damage robustly on the surface of the vehicle (Kaiming He *et al.*, 2016).

Based on the findings of Tsung-Yi Lin *et al.*, we know that FPN creates a multi-scale representation. It combines strong semantic features from the top layers with high-resolution details from the bottom layers. This allows the model to see both the sharp details of a scratch and the shape of a dent simultaneously (Tsung-Yi Lin *et al.*, 2017).

4.3.2 Region Proposal Network

The Region Proposal Network slides a small window over the feature maps generated by the Feature Pyramid Network to identify areas that might contain objects regardless of their damage class.

The fundamental job of RPN is to predict a binary probability of whether there is vehicle damage or background. It also makes rough adjustments to standard anchor boxes to better fit the potential object. Only the highest-scoring RoIs are passed to the next stage. A significant computational load is achieved through this filtering method (Shaoqing Ren *et al.*, 2015).

4.3.3 The Prototypical Classification Head

This is the core customisation of the Mask R-CNN architecture. Classification is performed by a linear layer ($Score = Wx + b$) in standard Mask R-CNN, which attempts to draw hyperplanes between classes (Kaiming He *et al.*, 2017). We have replaced this hyperplane approach with a Prototypical Box Predictor that is highly inspired by Jiawei Wu, Wenhan Xiong and William Yang Wang's Metric Learning. Instead of drawing planes, the model learns a vector space (Jiawei Wu, Wenhan Xiong and William Yang Wang, 2019).

The model learns a specific vector for each class. The vectors are also regarded as prototypes. These prototypes act as centroids or anchors in the high-dimensional feature space.

Mathematical Formulation for a detected region with a feature vector x and a class prototype P_k . The system first performs L_2 Normalisation, which projects all vectors onto a unit hypersphere. This normalisation ensures that the direction of the feature is based on the angular alignment instead of length.

$$\hat{x} = \frac{x}{\|x\|_2}, \hat{P}_k = \frac{P_k}{\|P_k\|_2}$$

The model calculates the similarity between the region and the class prototype using the Negative Squared Euclidean Distance. If the distance is small, the score is high, which indicates the region looks like the prototype.

$$S_k = -\alpha \cdot \|\hat{x} - \hat{P}_k\|_2^2$$

Here, the scaling factor α Amplifies the differences. A slightly smaller distance results in a much higher score after scaling. This forces the probability distribution to accelerate convergence and improve decision confidence.

4.3.4 Box and Mask Heads

Once the class is identified utilising the Prototypical head, two parallel branches finalise the output:

1. Box Regressor: A linear layer that predicts the precise coordinate offsets, also fine-tunes the proposal from the Region Proposal Network to tightly enclose the damaged region.
2. Mask Head: This branch utilises a Fully Convolutional Network. It takes the localised features and generates a 28×28 binary grid. This predicts the exact pixel-based shape of the damage, which separates the damaged surface from the surrounding vehicle surface.

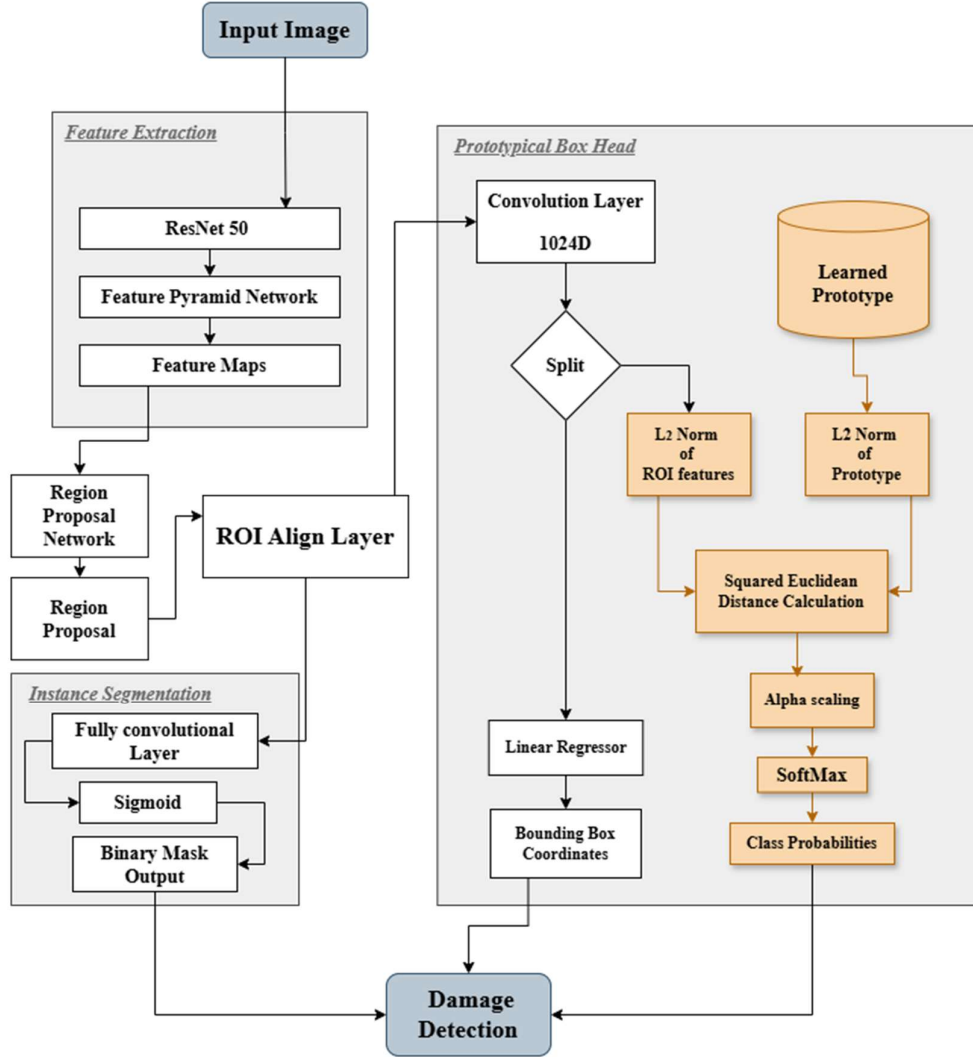


Figure 8 Prototypical Mask R-CNN architecture

Here, Figure 8 illustrates the complete data flow of the proposed Prototypical Mask R-CNN architecture. The input image is processed by the ResNet-50 backbone and Feature Pyramid Network (FPN) to generate feature maps. These maps serve as input for the Region Proposal Network (RPN), which identifies regions that contain damage classes. The regions are then fed into the Region of Interest Align layer. This layer extracts fixed-size feature vectors from the feature maps to ensure pixel-accurate alignment for segmentation. The aligned features are then flattened and passed through two fully connected layers to generate a 1024-dimensional representation. The Vector is then split into two parallel streams. One is the Regression stream, and another is the prototypical stream.

The standard linear regressor refines the bounding box coordinates from the CarDD dataset, and the prototypical stream replaces the standard classifier from the Mask R-CNN (Kaiming He *et al.*, 2017). Both the 1024-dimensional ROI features and the learned class prototypes undergo L_2 Normalisation to project them onto a unit hypersphere. Afterwards Squared Euclidean distances between the features and the prototypes have been calculated. The resulting distances are then scaled by α and passed through a SoftMax activation function to return final class probabilities.

In parallel, the Instance Segmentation branch receives the spatial features and utilises a fully convolutional layer. Then, a Sigmoid activation function is applied to generate a decoupled Binary Mask Output for the specific Vehicle Damage Class.

The final Damage detection output combines the refined coordinates from the carDD dataset, the predicted class label, and the segmentation mask.

4.4 Evaluation Metrics

To ensure the model’s reliability in a real-world scenario, the model is evaluated using standard industry benchmarks and architecture-specific diagnostic metrics.

4.4.1 COCO Mean Average Precision (mAP)

The primary metric used for evaluation of this project is the Mean Average Precision (mAP) as defined in the Microsoft COCO challenge (Tsung-Yi Lin *et al.*, 2015).

This metric calculates the average precision across 10 different Intersection over Union (IoU) thresholds, ranging from 0.50 to 0.95 in steps of 0.05. IoU measures how much the predicted bounding box overlaps with the ground truth. An IoU of 1.0 is a perfect match, and an IoU of 0.0 means no overlap (Tsung-Yi Lin *et al.*, 2015).

In vehicle exterior damage assessment, the model must accurately quantify the extent and the area of the damage to classify the damage types correctly. For example, A stricter threshold of IoU=0.90 may penalise the model if the predicted box is too loose or cuts off part of the damage, which ensures the model learns accurate localisation.

4.4.2 AP@50 (PASCAL VOC Standard)

We also testified AP@50 based on the Pascal Visual Object Classes (VOC) challenge by Mark Everingham *et al.* which measures the average precision where the predicted box overlaps at least 50% with the ground truth. It counts a prediction as “correct” if it captures at least half of the vehicle’s damaged area (Mark Everingham *et al.*, 2010).

This evaluation metric serves as a detection diagnostic. Here, mAP (0.50:0.95) measures localisation quality, and AP@50 measures localisation success.

So, if AP@50 is high but mAP is low, it indicates the model correctly identifies the damage type but struggles to draw tight boundaries around it. However, if AP@50 is low, it indicates the model is struggling to recognise the damage.

4.4.3 Prototype Separability

Since this architecture utilises a Prototypical Head, we have implemented Pairwise Prototype Distance as an evaluation metric because, based on the theory of Prototypical Networks. A classifier should learn features where different classes are geometrically distinct (Jake Snell, Kevin Swersky and Richard Zemel, 2017).

Here, we calculate the Squared Euclidean Distance between the learned centroid vectors of every class in the high-dimensional feature space. On a unit hypersphere, a squared distance of 2.0 implies the class vectors are orthogonal, which is 90° apart.

This metric confirms that the model is not confused. For example, if the distance between “Scratch” and “Crack” is near 0, the model views them as identical. A distance near 2.0 confirms the model has learned unique, separable visual features for each damage type, which ensures robust generalisation.

Chapter 5: Experimental Results

5.1 Training Dynamics and Convergence

We have monitored the model’s learning behaviour over 20 epochs to assess the stability of the Squared Euclidean distance-based Prototypical Head.

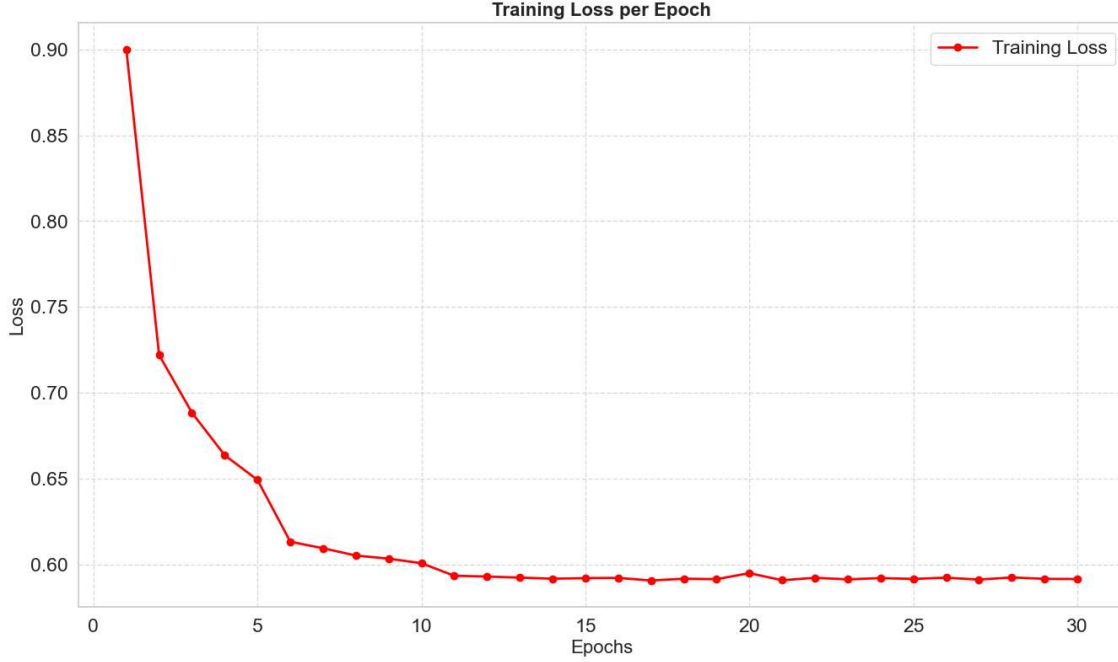


Figure 9 Training Loss Curve

The training loss in Figure 9 presents a convergence profile that validates our optimisation strategy. We have seen rapid adaptation from epoch 1 to 5. The loss drops quickly from 0.90 to 0.65, indicating the effectiveness of pre-trained ResNet-50 features, and they are being rapidly realigned to the damage detection task.

A distinct elbow occurs at Epoch 6. This correlates with the first step of the Learning Rate Scheduler of Lutz Prechelt’s work where the rate decayed from 0.001 to 0.0001. This optimisation allowed the model to settle into a deeper local minimum (Lutz Prechelt, 2002).

The loss stabilises around 0.59 from Epoch 11 onwards without diverging. This is a substantial finding. The smoothness of this curve confirms that our Hypersphere Constraint, combined with Temperature Scaling $\alpha = 20$ provided a stable optimisation landscape.

5.2 Quantitative Performance (mAP)

We evaluated the segmentation performance using the MS-COCO standard Mean Average Precision (mAP) (Tsung-Yi Lin *et al.*, 2015).

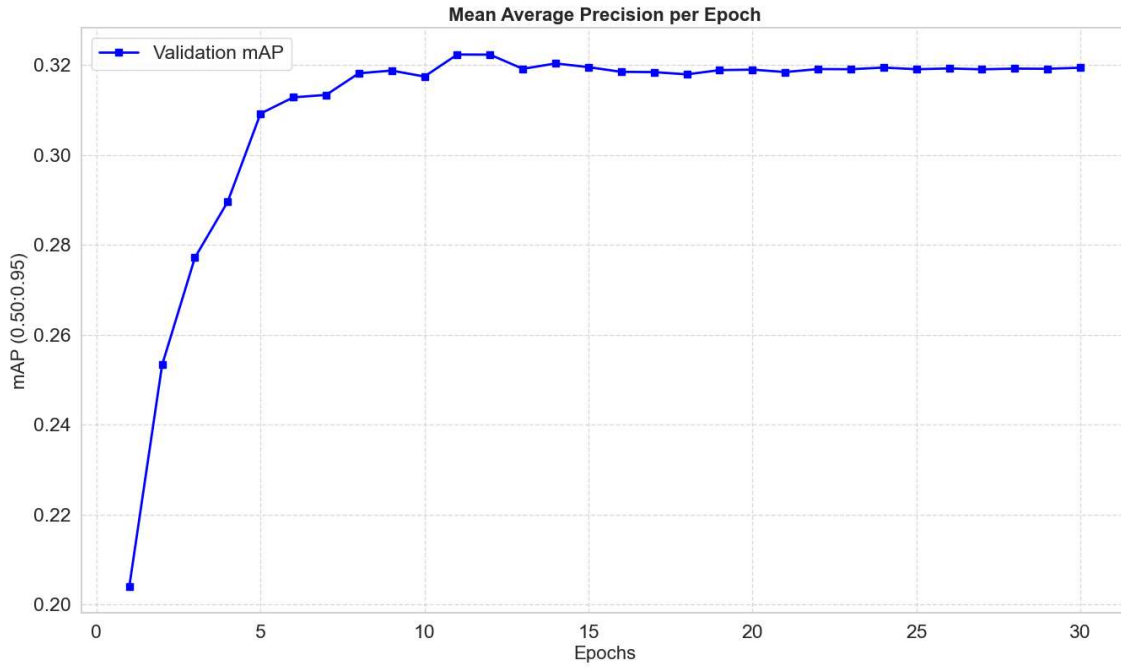


Figure 10 Validation curve

The Validation mAP mirrors the loss trajectory, confirming that minimising the Squared Euclidean distance in the embedding space directly translates to better detection performance.

The model reaches a peak mAP of approximately 32.2% by Epoch 11. We can observe a brief oscillation around Epoch 10. The model experiences a brief fluctuation before immediately recovering to reach its absolute maximum at Epoch 11.

The curve plateaus rather than declining in the final epochs (15 to 30), indicating that the model did not overfit, despite the 4,000 images of the skewed dataset of CarDD by (Xinkuang Wang, Wenjing Li and Zhonhcheng Wu, 2023). This resilience to overfitting is the key benefit of the Prototypical approach, which relies on class centroids rather than fitting every individual training example.

5.3 Detailed Metrics Evaluation

Table 1 Validation performance Metrics of prototypical Mask R-CNN on the CarDD dataset

Metric	Threshold	Value
mAP	IoU = 0.50:0.95	0.319
AP	IoU = 0.50	0.630
AR	MaxDets = 100	0.436
F1-Score	$\frac{2 \cdot (0.630 \cdot 0.436)}{0.630 + 0.436}$	0.515
AP (Small)	Area < 32 ²	0.074
AP(Large)	Area < 96 ²	0.320

Table 1 presents the comprehensive performance metrics evaluated on the 810 validation images. The model achieved a Mean Average Precision (mAP) of 0.319, which is a competitive result for the challenging CarDD dataset, particularly given the reliance on a ResNet-50 backbone rather than heavier transformers. I also calculated the F1-Score based on the AP@50 (0.515) and Recall (0.436) to provide a complete picture.

The model achieved an AP@50 of nearly 51%. This indicates that while the pixel-based segmentation (mAP 0.50:0.95) is challenging, the model is effective at locating and classifying the damage.

There is a significant performance disparity based on damage size.

- The model successfully performs large damages (AP=0.319).
- However, performance drops significantly for small objects (AP=0.074).
- Despite utilising a Feature Pyramid Network (FPN), the model still struggles to resolve defects smaller than 32 pixels. This confirms that “Small Damage Detection” remains the primary bottleneck.

Recall and Safety (AR = 0.436): The Average Recall of 0.436 indicates that the model successfully retrieves nearly half of all ground truth instances. Given the long-tailed nature of the CarDD dataset, where many samples are rare or ambiguous and this level of recall suggests that the Prototypical Head successfully prevented the model from collapsing into predicting only the majority vehicle damage class.

5.4 Feature Space Analysis: The Geometry of Learned Prototypes.

To validate the hypothesis that the Prototypical Head learns distinct and separable representations for each damage category, we visualised the pairwise Squared Euclidean Distance Matrix of the learned class prototypes (p_k).

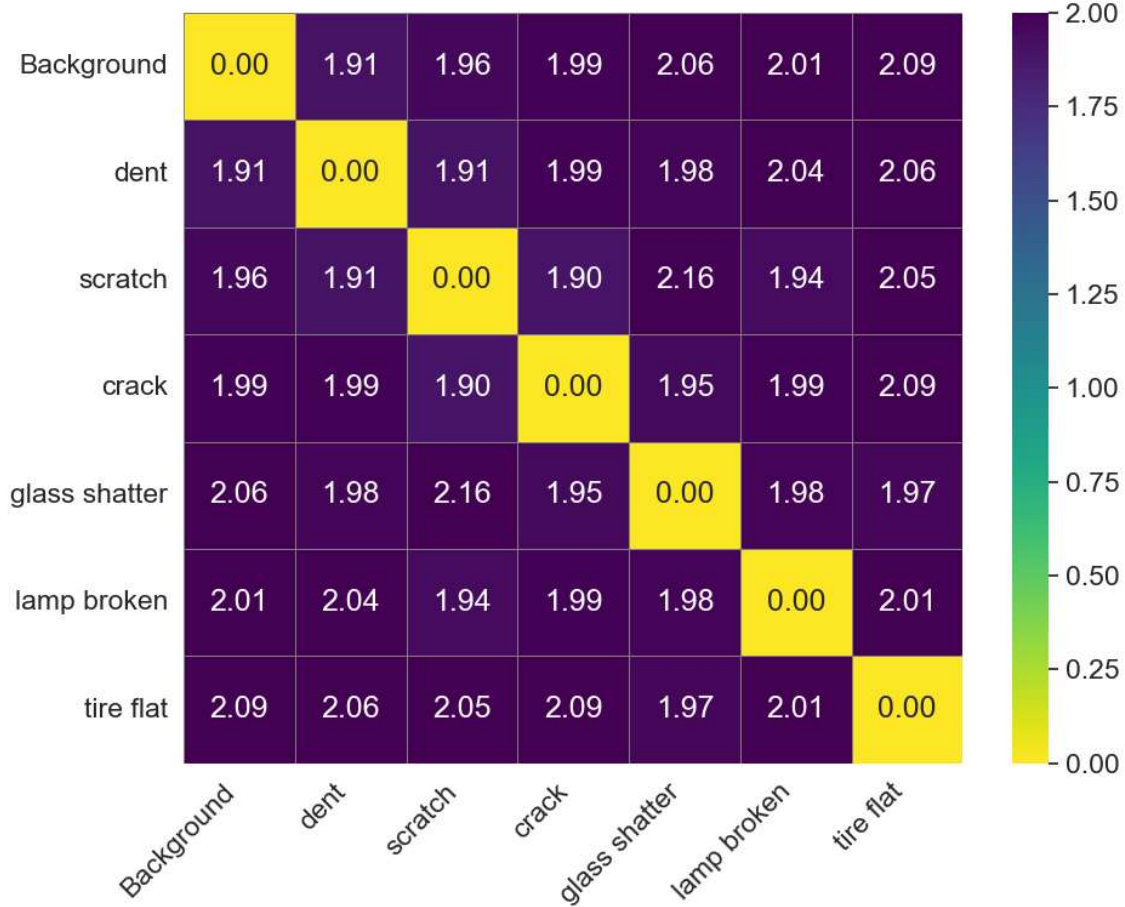


Figure 11 Prototype Similarity

The heatmap in Figure 11 visualises the Squared Euclidean distance between class centroids. The diagonal values of 0.00 represent self-distance. The off-diagonal values cluster tightly around 2.00. Since features are projected onto the unit hypersphere, a squared distance of 2.00 corresponds to 90° separation. This confirms that the Prototypical head has successfully learned to separate the defect categories into non-overlapping regions of the latent space.

As derived in our Methodology, our prototypes are normalised to the unit hypersphere ($\|p\| = 1$). The Squared Euclidean distance between two prototypes is related to the cosine of the angle θ between them:

$$\|p_i - p_j\|^2 = 2 - 2\cos(\theta)$$

In Figure 11, the off-diagonal elements in the heatmap consistently range between 1.91 and 2.09. This confirms that the model has learned to position the class centroids approximately 90 degrees apart in the 1024-dimensional vector space. This “Orthogonality” is the ideal state for a classifier, as it maximises the separation margin between conflicting classes.

To exemplify, the distance between “Dent” and “Scratch” is 1.91 and “Dent” vs “Lamp Broken” is 2.09. This suggests that the model perceives a slight semantic overlap between dents and scratches since both are body panel defects, whereas it sees “Lamp Broken” as a completely distinct damage event.

The distance between “Background” and all damage classes is consistently high. This indicates the model is highly effective at distinguishing “Damage” from noisy background within images, which is critical for reducing False Positives.

5.4.1 Geometric Derivation

We review the geometry of the embedding space to comprehend why the off-diagonal elements in our heatmap cluster are around 2.00. In our model architecture, we strictly enforced a Hypersphere Constraint. This means every prototype vector p was normalised to have a length of exactly $p = 1$. We can derive the relationship between the Squared Euclidean Distance and the Angle between classes by expanding the distance equation step-by-step.

Here,

$$d^2(p_i, p_j) = \|p_i - p_j\|^2$$

Expanding this using the dot product identity $\|v\|^2 = v \cdot v$:

$$\begin{aligned} &= (p_i \cdot p_i) - (p_i \cdot p_j) - (p_j \cdot p_i) + (p_j \cdot p_j) \\ &= (p_i \cdot p_i) - 2(p_i \cdot p_j) + (p_j \cdot p_j) \end{aligned}$$

And since vectors are normalised ($\|p_i\|^2 = 1$ and $\|p_j\|^2 = 1$):

$$\begin{aligned} &= 1 - 2(p_i \cdot p_j) + 1 \\ &= 2 - 2(p_i \cdot p_j) \end{aligned}$$

Applying the geometric definition of the dot product ($a \cdot b = \|a\| \cdot \|b\| \cos(\theta)$):

$$\|p_i - p_j\|^2 = 2 - 2\cos(\theta)$$

5.4.2 Interpretation of the Heatmap Values

Using the solved equation, we can now interpret Distance values into geometric angles:

1. If Distance = 0.00 (The Diagonal):

$$\begin{aligned} 2 - 2\cos(\theta) &= 0 \\ \Rightarrow \cos(\theta) &= 1 \\ \Rightarrow \theta &= 0^\circ \end{aligned}$$

Here, a distance of zero implies a perfect match. In the context of a confusion matrix, the diagonal 0.00 confirms that the prototype of a certain damage class is perfectly identical to itself.

2. If Distance = 2.00 (The Off-Diagonals):

$$\begin{aligned} 2 - 2 \cos(\theta) &= 2 \\ \Rightarrow -2 \cos(\theta) &= 0 \\ \Rightarrow \cos(\theta) &= 0 \\ \Rightarrow \theta &= 90^\circ \end{aligned}$$

Here, a distance of 2.00 implies the vectors are perpendicular. In high-dimensional space, orthogonality is the mathematical definition of unrelated features according to Pentti Kanerva. It means the model has successfully learned to recognise a certain feature completely distinct from other features (Pentti Kanerva, 2009).

3. If Distance = 4.00 (The Maximum):

$$\begin{aligned} 2 - 2 \cos(\theta) &= 4 \\ \Rightarrow \cos(\theta) &= -1 \\ \Rightarrow \theta &= 180^\circ \end{aligned}$$

This would indicate that the classes are entirely negatively correlated.

The fact that our heatmap values consistently range between 1.91 and 2.09 proves that the model has converged. The Prototypical Head has successfully pushed the 1024-Dimensional feature vector of every damage class into its own non-overlapping quadrant. The lowest distance (1.91) is between “Dent” and “Scratch”. This makes intuitive sense that both are deformations on the car body panel which share some visual features. On the contrary, “Lamp Broken” is further away (2.09), indicative of the fact that glass textures have distinct features that do not resemble body panels. Crucially, the distance between “Background” and all damage classes is high, ensuring the model’s reliability.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

This research set out to address the persistent challenge of Vehicle exterior damage assessment—a task historically plagued by visual ambiguity, class imbalance, and the difficulty of distinguishing similar defects like “scratches” and “cracks” under uncontrolled lighting conditions. By integrating Metric Learning principles into a standard two-stage detector, we successfully developed and validated a Prototypical Mask R-CNN architecture.

The core innovation of this study was the replacement of the traditional linear classification head with a Squared Euclidean Prototypical Head. Our experimental results show that this architectural shift offers significant advantages for damage detection tasks.

The most significant achievement is the structural organisation of the learned feature space. As evidenced by the prototype distance matrix, the model successfully converged to a state of orthogonality. This proves that the Prototypical Head forces the network to learn distinct feature representations that do not overlap for every damage category, which effectively solves the inter-class similarity problem that confuses standard classifiers.

Our implementation utilising Temperature Scaling ($\alpha = 20$) demonstrated smooth, monotonic convergence. The model achieved a peak mean Average Precision of 0.319 and an AP@50 of 0.630, indicating high reliability in localising damage.

The system showed resilience against the long-tailed distribution of the Car Damage Dataset. By relying on class centroids rather than decision boundaries, the model maintained an Average Recall (AR = 0.436) even for underrepresented classes like “Tire Flat”, avoiding the common pitfall of collapsing into predicting only the majority class.

However, the study also clearly outlined the limitations of the current ResNet-50-based architecture. The significant performance gap between large objects and small objects highlights a decisive bottleneck in resolution preservation. While the Feature Pyramid Network mitigated this to an extent, it is evident that detecting microscopic defects requires more specialised mechanisms.

6.2 Future Work

The Prototypical Mask R-CNN demonstrated robust convergence and successfully learned an orthogonal feature space for car damage categories, yet several limitations remain. This study has highlighted specific gaps regarding small object detection and the handling of extreme class imbalance. Future work should prioritise work on bridging the gap between this academic proof of concept and a deployment-ready insurance assessment tool.

Moreover, the Prototypical Head is mathematically robust but semantically limited. It cannot explain whether the damage is classified as “Severe” or describe its potential safety impact. Tianshuai Hu et al. discussed the emergence of VLA frameworks for autonomous systems, and this VLA framework, combined with projecting the learned embeddings from our damage detector into the latent space of a Multimodal Large Language Model (MLLM), the system could move beyond simple class labels (Tianshuai Hu *et al.*, 2025). It could generate descriptive safety reports and potentially bridge the gap between visual detection and actionable insurance intelligence.

References

- Adam Paszke *et al.* (2019) “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*. Curran Associates, Inc. Available at: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.
- Antonio Torralba and Alexei A. Efros (2011) “Unbiased Look at Dataset Bias,” in. IEEE. Available at: <https://doi.org/https://doi.org/10.1109/CVPR.2011.5995347>.
- Chris Tofallis (2009) *Least Squares Percentage Regression*. Available at: <https://doi.org/https://dx.doi.org/10.2139/ssrn.1406472>.
- Connor Shorten and Taghi M. Khoshgoftaar (2019) “A survey on Image Data Augmentation for Deep Learning,” *Journal of Big Data*, 6(1). Available at: <https://doi.org/10.1186/s40537-019-0197-0>.
- Deniz Cultu *et al.* (2022) *Claims 2030: Dream or reality?* Available at: <https://www.mckinsey.com/industries/financial-services/our-insights/claims-2030-dream-or-reality>.
- Jake Snell, Kevin Swersky and Richard Zemel (2017) “Prototypical Networks for Few-shot Learning,” in *Advances in Neural Information Processing Systems*. Curran Associates, Inc. Available at: https://proceedings.neurips.cc/paper_files/paper/2017/hash/cb8da6767461f2812ae4290eac7cbc42-Abstract.html.
- Jiawei Wu, Wenhan Xiong and William Yang Wang (2019) “Learning to Learn and Predict: A Meta-Learning Approach for Multi-Label Classification.” Available at: <http://arxiv.org/abs/1909.04176>.
- Kaiming He *et al.* (2016) *Deep Residual Learning for Image Recognition*. Available at: <https://doi.org/https://doi.org/10.1109/CVPR.2016.90>.
- Kaiming He *et al.* (2017) *Mask R-CNN*. Available at: <https://doi.org/https://doi.org/10.48550/arXiv.1703.06870>.
- Kalpesh Patil *et al.* (2017) “Deep learning based car damage classification,” in *Proceedings - 16th IEEE International Conference on Machine Learning and Applications, ICMLA 2017*. Institute of Electrical and Electronics Engineers Inc., pp. 50–54. Available at: <https://doi.org/10.1109/ICMLA.2017.0-179>.
- Kemal Oksuz *et al.* (2021) “Imbalance Problems in Object Detection: A Review,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*. IEEE Computer Society, pp. 3388–3415. Available at: <https://doi.org/10.1109/TPAMI.2020.2981890>.
- Li Liu *et al.* (2020) “Deep Learning for Generic Object Detection: A Survey,” *International Journal of Computer Vision*, 128(2), pp. 261–318. Available at: <https://doi.org/10.1007/s11263-019-01247-4>.

Luciano Da Fontoura Costa. and Roberto Marcond Cesar Jr. (2010) *Shape Analysis and Classification : Theory and Practice*. 1st ed. CRC Press. Available at: <https://doi.org/https://doi.org/10.1201/9781420037555>.

Lutz Prechelt (2002) “Early Stopping - But When,” in *Neural Networks: Tricks of the trade*. Springer, Berlin, Heidelberg, pp. 55–59. Available at: https://doi.org/https://doi.org/10.1007/3-540-49430-8_3.

Mark Everingham *et al.* (2010) “The pascal visual object classes (VOC) challenge,” *International Journal of Computer Vision*, 88(2), pp. 303–338. Available at: <https://doi.org/10.1007/s11263-009-0275-4>.

Pentti Kanerva (2009) *Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors*. Available at: <https://doi.org/https://doi.org/10.1007/s12559-009-9009-8>.

Ross Girshick (2015) “Fast R-CNN,” in *2015 IEEE International Conference on Computer Vision (ICCV)*. IEEE. Available at: <https://doi.org/https://doi.org/10.1109/ICCV.2015.169>.

Shaoqing Ren *et al.* (2015) “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” *Advances in Neural Information Processing Systems*, 28. Available at: <https://doi.org/https://doi.org/10.1109/TPAMI.2016.2577031>.

Tianshuai Hu *et al.* (2025) “Vision-Language-Action Models for Autonomous Driving: Past, Present, and Future.” Available at: <http://arxiv.org/abs/2512.16760>.

Tsung-Yi Lin *et al.* (2015) “Microsoft COCO: Common Objects in Context,” pp. 740–755. Available at: https://doi.org/https://doi.org/10.1007/978-3-319-10602-1_48.

Tsung-Yi Lin *et al.* (2017) *Feature Pyramid Networks for Object Detection*. Available at: <https://doi.org/https://doi.org/10.48550/arXiv.1612.03144>.

Tsung-Yi Lin *et al.* (2018) *Focal Loss for Dense Object Detection*. Available at: <https://doi.org/https://doi.org/10.48550/arXiv.1708.02002>.

Xiaopeng Yan *et al.* (2019) “Meta R-CNN : Towards General Solver for Instance-level Low-shot Learning,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE. Available at: <https://doi.org/https://doi.org/10.1109/ICCV.2019.00967>.

Xinkuang Wang, Wenjing Li and Zhongheng Wu (2023) “CarDD: A New Dataset for Vision-Based Car Damage Detection,” *IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS*, 24, pp. 7202–7214. Available at: <https://doi.org/https://doi.org/10.1109/TITS.2023.3258480>.

Yann Lecun, Yoshua Bengio and Geoffrey Hinton (2015) “Deep learning,” *Nature*. Nature Publishing Group, pp. 436–444. Available at: <https://doi.org/10.1038/nature14539>.

Ziwei Liu *et al.* (2019) *Large-Scale Long-Tailed Recognition in an Open World*.
Available at: <https://doi.org/https://doi.org/10.1109/CVPR.2019.00264>.

Appendix

1. Development Environment

- Framework: PyTorch with torchvision.
- Hardware: NVIDIA RTX 3060 (12GB VRAM). The 12GB limit required specific optimisations (Batch Size = 4).
- Precision: We utilised torch.cuda.amp.GradScaler for automatic Mixed Precision. This keeps the master weights in FP32 but performs the heavy matrix multiplications in FP16. This enables the heavy ResNet-50 backbone to fit in memory without sacrificing accuracy (Adam Paszke *et al.*, 2019).

2. Training Protocol

- Transfer Learning: The backbone was initialised with weights pre-trained on MS-COCO. COCO weights transfer the knowledge of how an object looks like. This allows the model to focus on learning what damage looks like.
- Optimiser: Stochastic Gradient Descent (SGD) with Momentum helps the optimiser navigate the ravines of the loss landscape and dampens oscillations.
- Learning Rate Schedule: Initialised at 0.001. We utilised a Learning Rate scheduler, decaying the rate by a factor of 10 every 5 epochs. This strategy allows the model to make coarse adjustments early on and fine-tune the prototypes later in training.

3. Data Augmentation

To combat the Centre Bias and lighting variations:

- Random Horizontal Flip: Forces the model to learn that damage can appear in any orientation.
- Color Jitter: Randomly adjusts brightness, contrast, and saturation. This makes the model robust to different weather conditions (sunny vs. overcast) and different car paint colors.

4. External Libraries

- Torch
- Torchvision: Contains the ResNet-50-FPN backbone, Mask R-CNN architecture and image transformation function.
- Pycocotools: required for the standard COCO evaluation metrics.
- Matplotlib and seaborn: Used for plotting.
- pandas
- numpy
- tqdm: for showcasing progressbar during training
- pillow (PIL)
- pickle
- random
- math

5. Source Code:

Cell 1: Configuration

```
class Config:
    BASE_DIR: str = r"\\CarDD_COCO"
    TRAIN_DIR: str = os.path.join(BASE_DIR, "train2017")
    VAL_DIR: str = os.path.join(BASE_DIR, "val2017")
    TRAIN_ANN: str = os.path.join(BASE_DIR, "annotations", "instances_train2017.json")
    VAL_ANN: str = os.path.join(BASE_DIR, "annotations", "instances_val2017.json")

    BATCH_SIZE = 4
    NUM_EPOCHS = 30
    LEARNING_RATE = 0.001

    # PROTOTYPICAL SETTINGS
    PROTO_SCALE = 20.0

    NMS_THRESH = 0.25

    DEVICE = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')

cfg = Config()
print(f"Configuration Loaded. Using Device: {cfg.DEVICE}")
```

Cell 2: CarDD analysis

```
def analyze_dataset_and_tell_story(annotation_file):
    print(f"Loading annotations from: {annotation_file}")

    # 1. DATA PREPARATION
    coco = COCO(annotation_file)
    anns = coco.loadAnns(coco.getAnnIds())
    cats = coco.loadCats(coco.getCatIds())
    cat_map = {c['id']: c['name'] for c in cats}

    data = []
    print("Processing geometry stats...")

    for ann in anns:
        w, h = ann['bbox'][2], ann['bbox'][3]
        area = ann['area']
        cat_name = cat_map[ann['category_id']]

        ar = w / max(h, 1)
        cx = ann['bbox'][0] + w/2
        cy = ann['bbox'][1] + h/2

        data.append({
```

```

        "Category": cat_name,
        "Area": area,
        "sqrt_Area": np.sqrt(area),
        "Aspect_Ratio": ar,
        "Log_AR": np.log2(ar + 1e-6),
        "cx": cx,
        "cy": cy
    })

df = pd.DataFrame(data)

#Global Font Scaling for better readability
sns.set_theme(style="whitegrid", font_scale=1.5)

# PLOT 1: Damage Class Distribution
plt.figure(figsize=(16, 10))
ax = sns.countplot(y="Category", data=df, order=df['Category'].value_counts().index,
palette="viridis")

plt.xlabel("Count", fontsize=25, fontweight='bold')
plt.ylabel("Category", fontsize=25, fontweight='bold')
plt.tick_params(axis='both', labels=24)

for container in ax.containers:
    ax.bar_label(container, padding=5, fontsize=14, fontweight='bold')

plt.tight_layout()
plt.show()

# PLOT 2: Object Size Distribution (Sqrt Area)
plt.figure(figsize=(16, 10))
sns.histplot(data=df, x="sqrt_Area", hue="Category", element="step", bins=40, kde=True,
alpha=0.3)

plt.axvline(32, color='red', linestyle='--', linewidth=3, label='Small (<32px)')
plt.axvline(96, color='blue', linestyle='--', linewidth=3, label='Medium (<96px)')

plt.xlabel("Sqrt(Area) in Pixels", fontsize=26, fontweight='bold')
plt.ylabel("Frequency", fontsize=26, fontweight='bold')
plt.xticks(fontsize=25)
plt.yticks(fontsize=25)

plt.legend(title='Category', fontsize=20, title_fontsize=16, loc='upper right')

plt.tight_layout()
plt.show()
# PLOT 3: Aspect Ratio with N-counts (Aspect Ratio Analysis (Log2 Scale))
plt.figure(figsize=(16, 10))

```

```

counts = df['Category'].value_counts()
df['Label_with_Count'] = df['Category'].apply(lambda x: f'{x}\n(n={counts[x]})')

sns.boxplot(x="Log_AR", y="Label_with_Count", data=df, palette="coolwarm",
            order=df['Label_with_Count'].value_counts().index)

plt.axvline(0, color='black', linestyle='--', linewidth=3, label="Square (1:1)")

plt.xlabel("Log2 Ratio ( <0: Tall | 0: Square | >0: Wide )", fontsize=18, fontweight='bold')
plt.ylabel("")
plt.xticks(fontsize=25)
plt.yticks(fontsize=25)
plt.legend(fontsize=24)

plt.tight_layout()
plt.show()

# PLOT 4: Spatial Heatmap of Damage Centroids
plt.figure(figsize=(16, 14))

hb = plt.hexbin(df['cx'], df['cy'], gridsize=40, cmap='inferno', mincnt=1)

plt.xlabel("X Coordinate (Width)", fontsize=26, fontweight='bold')
plt.ylabel("Y Coordinate (Height)", fontsize=26, fontweight='bold')
plt.xticks(fontsize=25)
plt.yticks(fontsize=25)
plt.gca().invert_yaxis()

cb = plt.colorbar(hb, label='Count density')
cb.set_label('Count Density', size=18, weight='bold')
cb.ax.tick_params(labelsize=15)

plt.tight_layout()
plt.show()

```

Cell 3: Dataset Class

```

class CarDDFullDataset(Dataset):
    def __init__(self, root: str, annotation_file: str, transforms=None, train: bool = False):
        self.root = root
        self.coco = COCO(annotation_file)
        self.ids = list(sorted(self.coco.imgs.keys()))
        self.transforms = transforms
        self.train = train

        sorted_cats = sorted(self.coco.getCatIds())
        self.cat_to_label = {cid: i+1 for i, cid in enumerate(sorted_cats)}

    def __getitem__(self, index: int):

```

```

img_id = self.ids[index]
img_info = self.coco.loadImgs(img_id)[0]
img_path = os.path.join(self.root, img_info['file_name'])

try:
    img = Image.open(img_path).convert("RGB")
except Exception as e:
    return torch.zeros(3, 256, 256), None

ann_ids = self.coco.getAnnIds(imgIds=img_id)
coco_anns = self.coco.loadAnns(ann_ids)

boxes, labels, masks = [], [], []

for ann in coco_anns:
    xmin, ymin, w, h = ann['bbox']
    if w > 1 and h > 1:
        boxes.append([xmin, ymin, xmin + w, ymin + h])
        labels.append(self.cat_to_label[ann['category_id']])
        mask = self.coco.annToMask(ann)
        masks.append(mask)

if len(boxes) > 0:
    boxes = torch.as_tensor(boxes, dtype=torch.float32)
    labels = torch.as_tensor(labels, dtype=torch.int64)
    masks = torch.as_tensor(np.stack(masks), dtype=torch.uint8)
else:
    boxes = torch.zeros((0, 4), dtype=torch.float32)
    labels = torch.zeros((0,), dtype=torch.int64)
    masks = torch.zeros((0, img_info['height'], img_info['width']), dtype=torch.uint8)

if self.train and random.random() < 0.5:
    img = TF.hflip(img)
    if len(boxes) > 0:
        width, _ = img.size
        boxes[:, [0, 2]] = width - boxes[:, [2, 0]]
        masks = TF.hflip(masks)

target = {}
target["boxes"] = boxes
target["labels"] = labels
target["masks"] = masks
target["image_id"] = torch.tensor([img_id])
target["area"] = (boxes[:, 3] - boxes[:, 1]) * (boxes[:, 2] - boxes[:, 0]) if len(boxes) > 0
else torch.zeros(0)
target["iscrowd"] = torch.zeros((len(boxes),), dtype=torch.int64)

if self.transforms:
    img = self.transforms(img)

```

```

        return img, target

    def __len__(self):
        return len(self.ids)

    def get_transform(train: bool = False):
        transforms = []
        if train:
            transforms.append(T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2))
        transforms.append(T.ToTensor())
        return T.Compose(transforms)

    def collate_fn(batch):
        batch = list(filter(lambda x: x[1] is not None, batch))
        if len(batch) == 0: return tuple()
        return tuple(zip(*batch))

```

Cell 4: Prototypical Architecture

```

class PrototypicalBoxPredictor(nn.Module):
    def __init__(self, in_channels, num_classes, scale=20.0):
        super().__init__()
        self.scale = scale
        self.cls_prototypes = nn.Parameter(torch.randn(num_classes, in_channels))
        self.bbox_pred = nn.Linear(in_channels, num_classes * 4)

        nn.init.xavier_uniform_(self.cls_prototypes)
        nn.init.xavier_uniform_(self.bbox_pred.weight)
        nn.init.constant_(self.bbox_pred.bias, 0)

    def forward(self, x):
        x = F.normalize(x, p=2, dim=1)
        p = F.normalize(self.cls_prototypes, p=2, dim=1)

        dists = torch.cdist(x, p, p=2)

        # Square the Distance
        squared_dists = dists.pow(2)

        logits = -squared_dists * self.scale

        bbox_deltas = self.bbox_pred(x)
        return logits, bbox_deltas

    def get_prototypical_model(num_classes):
        model = maskrcnn_resnet50_fpn(weights="DEFAULT")

        model.roi_heads.nms_thresh = cfg.NMS_THRESH

```

```

in_features_box = model.roi_heads.box_predictor.cls_score.in_features
model.roi_heads.box_predictor = PrototypicalBoxPredictor(
    in_features_box, num_classes, scale=cfg.PROTO_SCALE
)
in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
hidden_layer = 256
model.roi_heads.mask_predictor = MaskRCNNPredictor(in_features_mask, hidden_layer,
num_classes)

return model

```

Cell 5: Training & Eval

```

def train_one_epoch(model, optimizer, data_loader, device, epoch, scaler):
    model.train()
    loss_total = []
    pbar = tqdm(data_loader, desc=f"Epoch {epoch+1}", unit="batch")

    for images, targets in pbar:
        images = list(image.to(device) for image in images)
        targets = [{k: v.to(device) for k, v in t.items()} for t in targets]

        with torch.cuda.amp.autocast():
            loss_dict = model(images, targets)
            losses = sum(loss for loss in loss_dict.values())

        loss_value = losses.item()

        if not math.isfinite(loss_value):
            print(f'Loss is {loss_value}, stopping.')
            sys.exit(1)

        optimizer.zero_grad()
        scaler.scale(losses).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
        scaler.step(optimizer)
        scaler.update()

        loss_total.append(loss_value)
        pbar.set_postfix({"loss": f"{loss_value:.4f}"})

    return np.mean(loss_total)

@torch.no_grad()
def evaluate_metrics(model, data_loader, device, silent=False):
    model.eval()
    model.to(device)
    coco_results = []

```

```

if not silent: print(f"Evaluating on {len(data_loader.dataset)} images...")
for images, targets in tqdm(data_loader, disable=silent, desc="Validating"):
    images = list(img.to(device) for img in images)
    outputs = model(images)
    outputs = [{k: v.cpu() for k, v in t.items()} for t in outputs]

    for target, output in zip(targets, outputs):
        image_id = target["image_id"].item()
        boxes = output["boxes"].tolist()
        scores = output["scores"].tolist()
        labels = output["labels"].tolist()

        for i, score in enumerate(scores):
            if score < 0.01: continue
            box = boxes[i]
            res = {
                "image_id": image_id,
                "category_id": labels[i],
                "bbox": [box[0], box[1], box[2]-box[0], box[3]-box[1]],
                "score": score
            }
            coco_results.append(res)

if not coco_results: return None

coco_gt = data_loader.dataset.coco
coco_dt = coco_gt.loadRes(coco_results)

f = io.StringIO()
with redirect_stdout(f if silent else sys.stdout):
    coco_eval = COCOeval(coco_gt, coco_dt, 'bbox')
    coco_eval.evaluate()
    coco_eval.accumulate()
    coco_eval.summarize()

    if not silent:
        ap_50 = coco_eval.stats[1]
        recall_main = coco_eval.stats[8]
        fl_score = 0.0
        if (ap_50 + recall_main) > 0:
            fl_score = 2 * (ap_50 * recall_main) / (ap_50 + recall_main)
        print("\n" + "="*40)
        print(f'mAP (IoU=0.50:0.95): {coco_eval.stats[0]:.4f}')
        print(f'AP (IoU=0.50): {ap_50:.4f}')
        print(f'F1 Score: {fl_score:.4f}')
        print("="*40)
    return coco_eval

```

Cell 6: Eval Viz

```

import matplotlib.pyplot as plt

```

```

import seaborn as sns
import torch
import torch.nn.functional as F
import random
from torchvision.utils import draw_bounding_boxes, draw_segmentation_masks

# 1. PLOT LOSS CURVE
def plot_loss_curve(loss_history):
    epochs = range(1, len(loss_history) + 1)
    plt.figure(figsize=(16, 9))
    plt.plot(epochs, loss_history, 'o-', color='red', linewidth=2, label='Training Loss')
    plt.title('Training Loss per Epoch', fontsize=16, fontweight='bold')
    plt.xlabel('Epochs', fontsize=16)
    plt.ylabel('Loss', fontsize=16)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.legend()
    plt.show()

# 2. PLOT mAP CURVE
def plot_map_curve(map_history):
    epochs = range(1, len(map_history) + 1)
    plt.figure(figsize=(16, 9))
    plt.plot(epochs, map_history, 's-', color='blue', linewidth=2, label='Validation mAP')
    plt.title('Mean Average Precision per Epoch', fontsize=16, fontweight='bold')
    plt.xlabel('Epochs', fontsize=16)
    plt.ylabel('mAP (0.50:0.95)', fontsize=16)
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.legend()
    plt.show()

# 3. PLOT PROTOTYPE DISTANCE HEATMAP
def plot_prototype_heatmap(model, dataset):
    print("Generating Prototype Distance Heatmap...")
    model.eval()

    # Extract and Normalize Prototypes
    prototypes = model.roi_heads.box_predictor.cls_prototypes.data.cpu()
    prototypes_norm = F.normalize(prototypes, p=2, dim=1)

    # Compute Squared Euclidean Distance Matrix
    # cdist (p=2) and then square it
    dist_matrix = torch.cdist(prototypes_norm, prototypes_norm, p=2).pow(2).numpy()

    # Generate Labels
    labels = ["Background"]
    label_to_name = {label: dataset.coco.cats[cid]['name'] for cid, label in
dataset.cat_to_label.items()}
    for i in range(1, len(labels) + len(label_to_name)):
        labels.append(label_to_name.get(i, f'Class {i}'))

```

```

plt.figure(figsize=(10, 8))

sns.heatmap(dist_matrix, xticklabels=labels, yticklabels=labels,
            annot=True, fmt=".2f", cmap="viridis_r", vmin=0.0, vmax=2.0,
            linewidths=0.5, linecolor='gray')

plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
# 4. PLOT IMAGE PREDICTIONS ONLY
def visualize_sample_predictions(model, dataset, device, num_samples=2):
    print(f"Visualizing {num_samples} Sample Predictions...")
    model.eval()

    # Generate Labels Map
    labels_map = ["Background"]
    label_to_name = {label: dataset.coco.cats[cid]['name'] for cid, label in
dataset.cat_to_label.items()}
    for i in range(1, len(labels_map) + len(label_to_name)):
        labels_map.append(label_to_name.get(i, f"Class {i}"))

    idxs = random.sample(range(len(dataset)), num_samples)

    for idx in idxs:
        img, target = dataset[idx]
        img_gpu = img.to(device).unsqueeze(0)

        with torch.no_grad():
            output = model(img_gpu)[0]

        img_int = (img * 255).to(dtype=torch.uint8)

        mask_idx = output['scores'] > 0.5
        pred_boxes = output['boxes'][mask_idx]
        pred_masks = output['masks'][mask_idx]
        pred_scores = output['scores'][mask_idx]
        pred_lbls_idx = output['labels'][mask_idx]

        pred_strings = [f"{labels_map[l.item()]}: {s.item():.2f}" for l, s in zip(pred_lbls_idx,
pred_scores)]

        res_img = img_int.cpu()

        # Draw Masks
        if len(pred_masks) > 0:
            bool_masks = pred_masks.squeeze(1) > 0.5
            res_img = draw_segmentation_masks(res_img, bool_masks.cpu(), alpha=0.5,
colors="red")

```

```

# Draw Boxes
if len(pred_boxes) > 0:
    res_img = draw_bounding_boxes(res_img, pred_boxes.cpu(), labels=pred_strings,
    colors="red", width=3)

# Plot Independent Figure
plt.figure(figsize=(16, 16))
plt.imshow(res_img.permute(1,2,0).numpy())
plt.title(f'Prediction Sample (Image ID: {target['image_id'].item()})', fontsize=14)
plt.axis("off")
plt.show()

```

Cell 7: all Execution

```

if __name__ == "__main__":
    # 0. Run Statistical Analysis (EDA)
    if os.path.exists(cfg.TRAIN_ANN):
        analyze_dataset_and_tell_story(cfg.TRAIN_ANN)

    if os.path.exists(cfg.TRAIN_DIR):
        # 1. Load Data
        train_ds = CarDDFullDataset(cfg.TRAIN_DIR, cfg.TRAIN_ANN,
        get_transform(train=True), train=True)
        val_ds = CarDDFullDataset(cfg.VAL_DIR, cfg.VAL_ANN,
        get_transform(train=False), train=False)

        train_loader = DataLoader(train_ds, batch_size=cfg.BATCH_SIZE, shuffle=True,
        collate_fn=collate_fn)
        val_loader = DataLoader(val_ds, batch_size=cfg.BATCH_SIZE, shuffle=False,
        collate_fn=collate_fn)

        # 2. Init Model
        num_classes = len(train_ds.cat_to_label) + 1
        model = get_prototypical_model(num_classes).to(cfg.DEVICE)

        # 3. Check for existing weights/history
        model_path = "latest_model.pth"
        loss_history = []
        map_history = []

        if os.path.exists(model_path) and os.path.exists("history.pkl"):
            print(f'\nFOUND EXISTING MODEL: {model_path}')
            model.load_state_dict(torch.load(model_path))
            with open("history.pkl", "rb") as f:
                data = pickle.load(f)
                loss_history = data['loss']
                map_history = data['map']
        else:
            # 4. Train

```

```

        params = [p for p in model.parameters() if p.requires_grad]
        optimizer = torch.optim.SGD(params, lr=cfg.LEARNING_RATE, momentum=0.9,
weight_decay=0.0005)
        lr_scheduler = torch.optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.1)
        scaler = torch.cuda.amp.GradScaler()

        print("Starting Training with Live Metrics Tracking...")
        for epoch in range(cfg.NUM_EPOCHS):
            avg_loss = train_one_epoch(model, optimizer, train_loader, cfg.DEVICE, epoch,
scaler)
            loss_history.append(avg_loss)
            lr_scheduler.step()

            # Silent eval for graph data
            eval_result = evaluate_metrics(model, val_loader, cfg.DEVICE, silent=True)
            current_map = eval_result.stats[0] if eval_result else 0.0
            map_history.append(current_map)

            print(f'Epoch {epoch} | Loss: {avg_loss:.4f} | Val mAP: {current_map:.4f}')

            torch.save(model.state_dict(), model_path)
            with open("history.pkl", "wb") as f:
                pickle.dump({'loss': loss_history, 'map': map_history}, f)
            print("Training Complete.")

# 5. Visualize
if len(loss_history) > 0:
    print("\n1. Training Loss Curve ")
    plot_loss_curve(loss_history)

    print("\n2. Validation mAP Curve ")
    plot_map_curve(map_history)

print("\n3. Detailed Metrics Evaluation")
evaluate_metrics(model, val_loader, cfg.DEVICE, silent=False)

print("\n4. PROTOTYPE DISTANCE HEATMAP")
plot_prototype_heatmap(model, val_ds)

print("\n5. Qualitative Predictions")
visualize_sample_predictions(model, val_ds, cfg.DEVICE, num_samples=3)

else:
    print("Error")

```